

UVM Project - SPI Slave with Single Port RAM

SPI – RAM – SPI WRAPPER

Mazen Mohamed Hemdan
Menna Alah Mohamed Farag Mahmoud

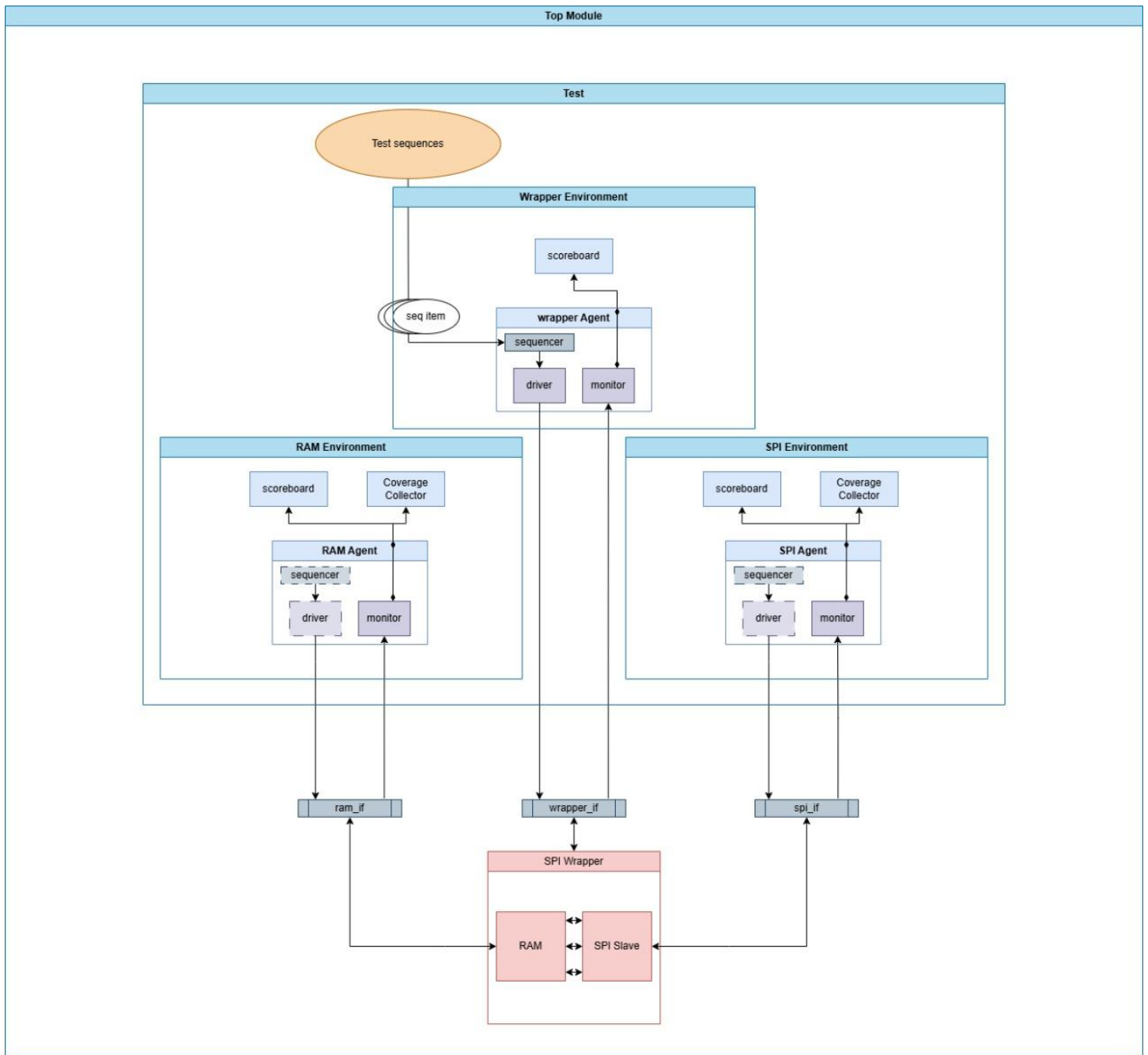
Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
RESET_1	When the reset is asserted the outputs should be low	Directed at the start of simulation then randomized to be deasserted most of the time	-	An assertion ensures that whenever reset is asserted, the outputs are low
SPI_WRITE_ADDR	When SS_n is asserted to begin communication and MOSI is send with the sequence 0 => 0 => 0 to start write address operation , the next MOSI bits are send as a write_address to the memory through rx_data then SS_n is deasserted to end communication	1. SS_n constrained to be deasserted every 13 cycles 2. MOSI bits are randomized under constrain that first 3 bits are 000	1. coverpoint on SS_n to capture full transaction duration 2. coverpoint on MOSI to validate transition 000 3. coverpoint on rx_data first to bits to check possible value 00	1. A checker in the scoreboard to check the output rx_data with a golden model output rx_data 2. An assertion to check that SS_n should eventually be deasserted to end communication 3. An assertion in design to check correct FSM transition IDLE -> CHK_CMD -> WRITE
SPI_WRITE_DATA	When SS_n is asserted to begin communication and MOSI is send with the sequence 0 => 0 => 1 to start write data operation , the next MOSI bits are send as a write_data to the memory through rx_data then SS_n is deasserted to end communication	1. SS_n constrained to be deasserted every 13 cycles 2. MOSI bits are randomized under constrain that first 3 bits are 001	1. coverpoint on SS_n to capture full transaction duration 2. coverpoint on MOSI to validate transition 001 3. coverpoint on rx_data first to bits to check possible value 01	1. A checker in the scoreboard to check the output rx_data with a golden model output rx_data 2. An assertion to check that SS_n should eventually be deasserted to end communication 3. An assertion in design to check correct FSM transition IDLE -> CHK_CMD -> WRITE
SPI_READ_ADDR	When SS_n is asserted to begin communication and MOSI is send with the sequence 1 => 1 => 0 to start read address operation , the next MOSI bits are send as a read_address to the memory through rx_data then SS_n is deasserted to end communication	1. SS_n constrained to be deasserted every 13 cycles 2. MOSI bits are randomized under constrain that first 3 bits are 110	1. coverpoint on SS_n to capture full transaction duration 2. coverpoint on MOSI to validate transition 110 3. coverpoint on rx_data first to bits to check possible value 10	1. A checker in the scoreboard to check the output rx_data with a golden model output rx_data 2. An assertion to check that SS_n should eventually be deasserted to end communication 3. An assertion in design to check correct FSM transition IDLE -> CHK_CMD -> READ_ADD
SPI_READ_DATA	When SS_n is asserted to begin communication and MOSI is send with the sequence 1 => 1 => 1 to start read data operation , the next MOSI bits are send as a dummy to the memory through rx_data then spi expects tx_data from memory and SS_n is deasserted to end communication	1. SS_n constrained to be deasserted every 23 cycles 2. MOSI bits are randomized under constrain that first 3 bits are 111	1. coverpoint on SS_n to capture extended transaction duration 2. coverpoint on MOSI to validate transition 111 3. coverpoint on rx_data first to bits to check possible value 11	1. A checker in the scoreboard to check the output rx_data with a golden model output rx_data 2. An assertion to check that SS_n should eventually be deasserted to end communication 3. An assertion in design to check correct FSM transition IDLE -> CHK_CMD -> READ_DATA
SPI_RX_VALID	after any valid operation when sending rx_data to RAM rx_valid should be asserted	-	-	An assertion that after any valid operation rx_valid must be asserted
SPI_TX_VALID	incase of read data operation tx_valid should be asserted to receive tx_data from RAM and send them serially through MISO bits	tx_valid constrained to be high incase of read data	-	-

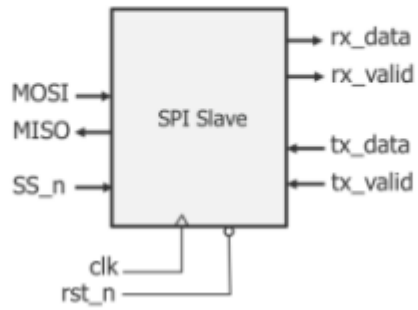
Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
RESET_1	When the reset is asserted the outputs should be low	Directed at the start of simulation then randomized to be deasserted most of the time	-	An assertion ensures that whenever reset is asserted, the outputs are low
RAM_WRITE	when rx_valid is high and din[9:8] bits are 00, din[7:0] bits are stored as write address, on the other hand when din[9:8] bits are 01, din[7:0] bits are stored as write data in the write address in the RAM	1. rx_valid constrained to be asserted most of the time 2. write address operation constrained to be followed by another write address or write data operations	1. coverpoint on din[9:8] bits to take values 00,01 2. coverpoint on din[9:8] transition from write address(00) => write data (01) 3. cross cover between rx_valid when it's high and din[9:8] values 00,01	1. An assertion to check that every write address should eventually be followed by write data
RAM_READ	when rx_valid is high and din[9:8] bits are 10, din[7:0] bits are stored as read address, on the other hand when din[9:8] bits are 11, [7:0] bits in the read address in the RAM are sent through dout, and tx_valid is asserted	1. rx_valid constrained to be asserted most of the time 2. read address operation constrained to be followed by read data operation	1. coverpoint on din[9:8] bits to take values 10,11 2. coverpoint on din[9:8] transition from read address(10) => read data (11) 3. cross cover between rx_valid when it's high and din[9:8] values 10,11 4. cross cover between din[9:8] value 11(read data) and tx_valid when it's high	1. A checker in the scoreboard to check the outputs dout and tx_valid with a golden model outputs dout and tx_valid 2. An assertion to check that every read address should eventually be followed by read data
RAM_WRITE_READ	both write and read operations are implemented correctly for RAM	1. write address constrained to be followed by another write address or write data 2. read address constrained to be followed by another read address or read data 3. after write data, next operation is either read address or write address with 60:40 ratio 4. after read data, next operation is either write address or read address with 60:40 ratio	1. coverpoint on din[9:8] to check write address(00) => write data(01) => read address(10) => read data(11) transition	1. A checker in the scoreboard to check the outputs dout and tx_valid with a golden model outputs dout and tx_valid 2. An assertion to check that tx_valid is deasserted during write and read address operations 3. An assertion to check that tx_valid is asserted during read data operation

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
RESET_1	When the reset is asserted the outputs should be low	Directed at the start of simulation then randomized to be deasserted most of the time	-	An assertion ensures that whenever reset is asserted, the outputs are low
WRAPPER_WRITE_ADDR	When SS_n is asserted to begin communication and MOSI is send with the sequence 0 => 0 => 0 to start write address operation , the next MOSI bits are send as a write_address then SS_n is deasserted to end communication	1. SS_n constrained to be deasserted every 13 cycles 2.MOSI bits are randomized under constrain that first 3 bits are 000 3.write address operation constrained to be followed by another write address or write data operations	-	An assertion to make sure MISO remains stable
WRAPPER_WRITE_DATA	When SS_n is asserted to begin communication and MOSI is send with the sequence 0 => 0 => 1 to start write data operation , the next MOSI bits are send as a write_data then SS_n is deasserted to end communication	1. SS_n constrained to be deasserted every 13 cycles 2.MOSI bits are randomized under constrain that first 3 bits are 001 3.write data operation constrained to be followed by read address or write address with ratio 60:40	-	An assertion to make sure MISO remains stable
WRAPPER_READ_ADDR	When SS_n is asserted to begin communication and MOSI is send with the sequence 1 => 1 => 0 to start read address operation , the next MOSI bits are send as a read_address then SS_n is deasserted to end communication	1. SS_n constrained to be deasserted every 13 cycles 2.MOSI bits are randomized under constrain that first 3 bits are 110 3.read address operation constrained to be followed by read data or read address	-	An assertion to make sure MISO remains stable
WRAPPER_READ_DATA	When SS_n is asserted to begin communication and MOSI is send with the sequence 1 => 1 => 1 to start read data operation , the next MOSI bits are send as a dummy and read data are out bit by bit through MISO then SS_n is deasserted to end communication	1. SS_n constrained to be deasserted every 23 cycles 2.MOSI bits are randomized under constrain that first 3 bits are 111 3.read data operation constrained to be followed by write address or read address with ratio 60:40	-	A checker in the scoreboard to check the output MISO with a golden model output MISO

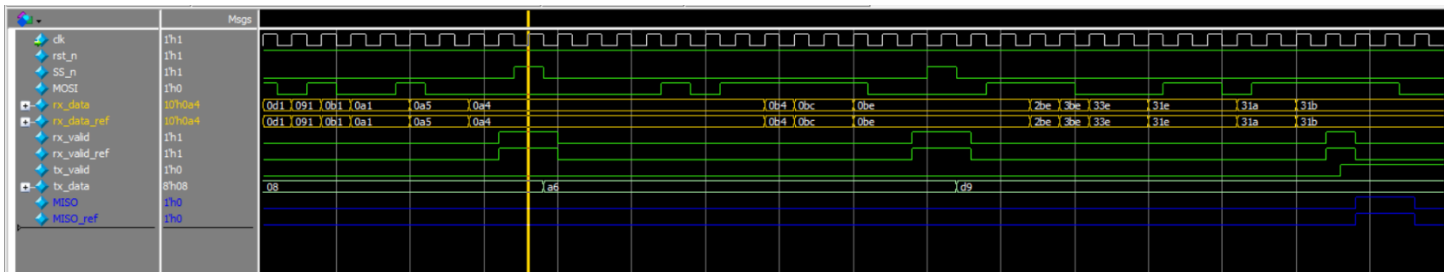
UVM Structure



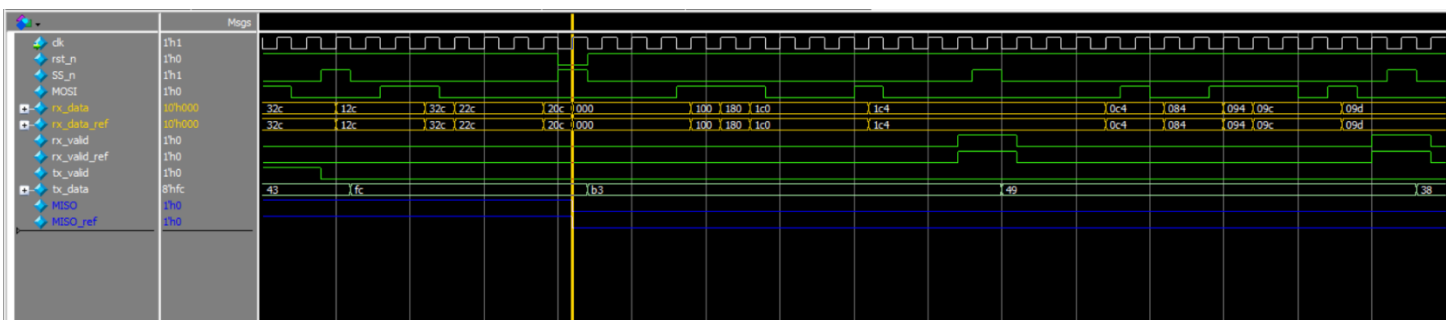
Part 1: UVM Environment for SPI-Slave



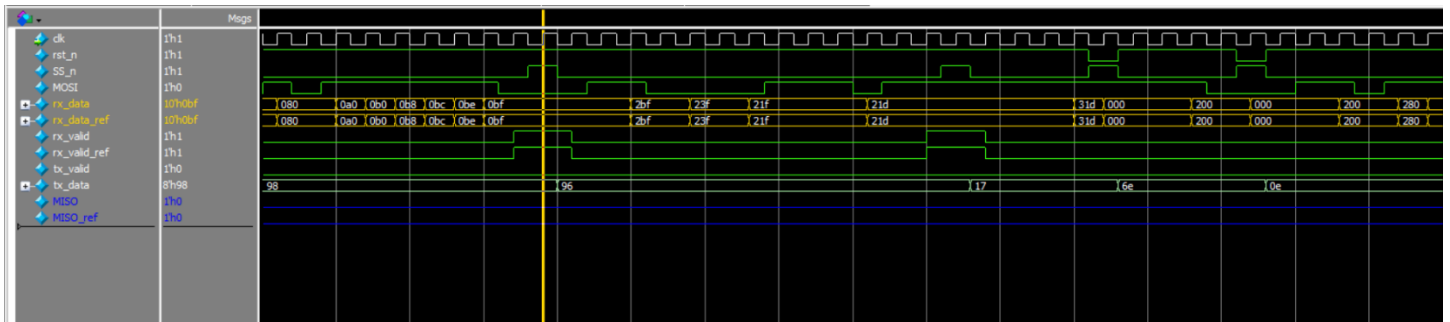
Write Addr Wave MOSI_bits[10:8] = 3'b000



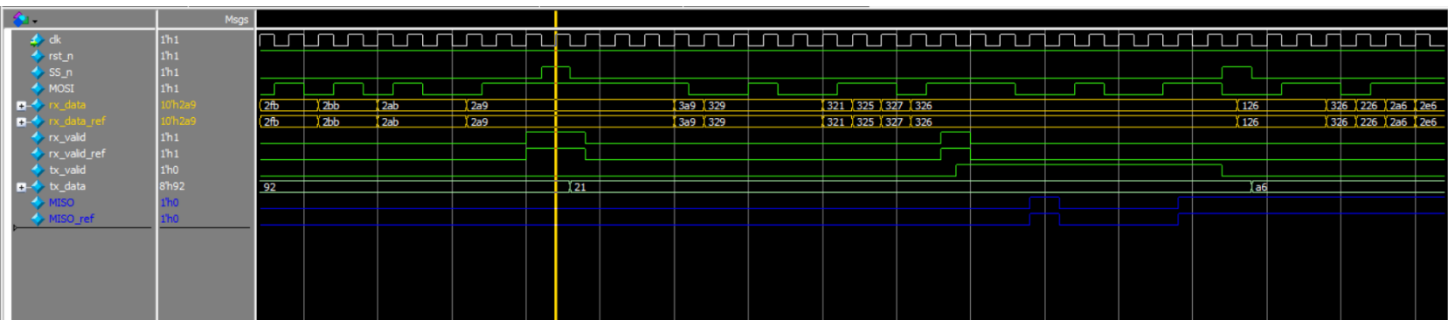
Write Data Wave MOSI_bits[10:8] = 3'b001



Read Addr Wave MOSI_bits[10:8] = 3'b110



Read Data Wave MOSI_bits[10:8] = 3'b111



2) Transcript Output

```
# ***** IMPORTANT RELEASE NOTES *****
#
# You are using a version of the UVM library that has been compiled
# with 'UVM_NO_DEPRECATED' undefined.
# See http://www.eda.org/svdb/view.php?id=3313 for more details.
#
# You are using a version of the UVM library that has been compiled
# with 'UVM_OBJECT_MUST_HAVE_CONSTRUCTOR' undefined.
# See http://www.eda.org/svdb/view.php?id=3770 for more details.
#
# (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test spi_test...
# UVM_INFO spi_test.sv(36) @ 0: uvm_test_top [run_phase] Reset Sequence Start
# *****
# * Questa UVM Transaction Recording Turned ON. *
# * recording_detail has been set. *
# * To turn off, set 'recording_detail' to off: *
# * uvm_config_db#(int) ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# *****
# UVM_INFO spi_test.sv(38) @ 2: uvm_test_top [run_phase] Reset Sequence End
# UVM_INFO spi_test.sv(39) @ 2: uvm_test_top [run_phase] Main Sequence Start
# UVM_INFO spi_test.sv(41) @ 100002: uvm_test_top [run_phase] Main Sequence End
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 100002: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# Final Counters for spi 50001, 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 8
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [run_phase] 4
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 100002 ns Iteration: 61 Instance: /top
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

3) UVM Factory

```
uvm printfactory
#
#### Factory Configuration (*)
#
#   No instance or type overrides are registered with this factory
#
# All types registered with the factory: 50 total
# (types without type names will not be printed)
#
#   Type Name
#   -----
#   config_obj
#   questa_uvm_recorder
#   spi_agent
#   spi_coveragecollector
#   spi_driver
#   spi_env
#   spi_main_sequence
#   spi_monitor
#   spi_reset_sequence
#   spi_scoreboard
#   spi_seq_item
#   spi_sequencer
#   spi_test
# (*) Types with no associated type name will be printed as <unknown>
#
####
#
```

4) Design Code (After Bugs Correction)

```
module SLAVE (spi_IF.DOUT spi_vif);
    localparam IDLE      = 3'b000;
    localparam CHK_CMD   = 3'b001; // change chk_cmd value from 2 to 1
    localparam WRITE      = 3'b010;
    localparam READ_ADD   = 3'b011;
    localparam READ_DATA  = 3'b100;

    reg [3:0] counter;
    reg      received_address;
    reg [2:0] cs, ns;

    always @(posedge spi_vif.clk) begin
        if (~spi_vif.rst_n) begin
            cs <= IDLE;
        end
        else begin
            cs <= ns;
        end
    end

    always @(*) begin
        case (cs)
            IDLE : begin
                if (spi_vif.SS_n)
                    ns = IDLE;
                else
                    ns = CHK_CMD;
            end
            CHK_CMD : begin
                if (spi_vif.SS_n)
                    ns = IDLE;
                else begin
                    if (~spi_vif.MOSI)
                        ns = WRITE;
                    else begin
                        if (!received_address) // Edit
                            ns = READ_ADD;
                        else
                            ns = READ_DATA;
                        end
                    end
                end
            end
            WRITE : begin
                if (spi_vif.SS_n)
                    ns = IDLE;
                else
                    ns = WRITE;
            end
            READ_ADD : begin
                if (spi_vif.SS_n)
                    ns = IDLE;
                else
                    ns = READ_ADD;
            end
            READ_DATA : begin
                if (spi_vif.SS_n)
                    ns = IDLE;
                else
                    ns = READ_DATA;
            end
        endcase
    end

    always @(posedge spi_vif.clk) begin
        if (~spi_vif.rst_n) begin
            spi_vif.rx_data <= 0;
            spi_vif.rx_valid <= 0;
            received_address <= 0;
            spi_vif.MISO <= 0;
        end
        else begin
            case (cs)
                IDLE : begin
                    spi_vif.rx_valid <= 0;
                end
                CHK_CMD : begin
                    counter <= 10;
                end
                WRITE : begin
                    if (counter > 0) begin
                        spi_vif.rx_data[counter-1] <= spi_vif.MOSI;
                        counter <= counter - 1;
                    end
                    else begin
                        spi_vif.rx_valid <= 1;
                    end
                end
                READ_ADD : begin
                    if (counter > 0) begin
                        spi_vif.rx_data[counter-1] <= spi_vif.MOSI;
                        counter <= counter - 1;
                    end
                    else begin
                        spi_vif.rx_valid <= 1;
                        received_address <= 1;
                    end
                end
                READ_DATA : begin
                    if (spi_vif.tx_valid) begin
                        spi_vif.rx_valid <= 0;
                        if (counter > 0) begin
                            spi_vif.MISO <= spi_vif.tx_data[counter-1];
                            counter <= counter - 1;
                        end
                    end
                    else begin
                        received_address <= 0;
                        counter <= 10;
                    end
                end
            endcase
        end
    end

    // SWA
    `ifdef SIM
endmodule
```

5) Assertions

```
// SVA
`ifdef SIM

    property idle_chk;
        @(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) (cs == IDLE) |-> (ns == CHK_CMD);
    endproperty
    a_1 : assert property(idle_chk);
    c_1 : cover property(idle_chk);

    property chk_write_read_add;
        @(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) (cs == CHK_CMD) |-> (ns == WRITE || ns == READ_DATA || ns == READ_ADD);
    endproperty
    a_2 : assert property(chk_write_read_add);
    c_2 : cover property(chk_write_read_add);

    property write_idle;
        @(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) ($past($past($past(spi_vif.SS_n))) && cs == WRITE) |-> ##12 (cs == IDLE);
    endproperty
    a_3 : assert property(write_idle);
    c_3 : cover property(write_idle);

    property read_add_idle;
        @(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) ($past($past($past(spi_vif.SS_n))) && cs == READ_ADD) |-> ##12 (cs == IDLE);
    endproperty
    a_4 : assert property(read_add_idle);
    c_4 : cover property(read_add_idle);

    property read_data_idle;
        @(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) ($past($past($past(spi_vif.SS_n))) && cs == READ_DATA) |-> ##21 (cs == IDLE);
    endproperty
    a_5 : assert property(read_data_idle);
    c_5 : cover property(read_data_idle);

`endif

endmodule
```

6) SVA Module

```
module spi_SVA(spi_IF.SVA s_vif);

    property rst_prp;
        @(posedge s_vif.clk) !s_vif.rst_n | => (!s_vif.MISO && !s_vif.rx_valid && ~s_vif.rx_data);
    endproperty
    assert1 : assert property(rst_prp);
    cover1 : cover property(rst_prp);

    sequence write_addr_sequence;
        s_vif.SS_n ##2 !s_vif.MOSI ##1 !s_vif.MOSI ##1 !s_vif.MOSI;
    endsequence
    sequence write_data_sequence;
        s_vif.SS_n ##1 !s_vif.MOSI ##1 !s_vif.MOSI ##1 s_vif.MOSI;
    endsequence
    sequence read_addr_sequence;
        s_vif.SS_n ##1 s_vif.MOSI ##1 s_vif.MOSI ##1 !s_vif.MOSI;
    endsequence
    sequence read_data_sequence;
        s_vif.SS_n ##1 s_vif.MOSI ##1 s_vif.MOSI ##1 s_vif.MOSI;
    endsequence
    sequence rx_ssn_sequence;
        ##10 s_vif.rx_valid && s_vif.SS_n[->1];
    endsequence
    property valid_sequence;
        @(posedge s_vif.clk) disable iff(!s_vif.rst_n)
        ((write_addr_sequence) or (write_data_sequence) or (read_addr_sequence) or (read_data_sequence) |-> rx_ssn_sequence);
    endproperty
    assert2 : assert property(valid_sequence);
    cover2 : cover property(valid_sequence);

endmodule
```


7) Golden Model Design

```
module SPI_SLAVE(clk,rst_n,SS_n,rx_data,rx_valid,tx_data,tx_valid,MOSI,MISO);
parameter IDLE = 3'b000;
parameter CHK_CMD = 3'b001;
parameter WRITE = 3'b010;
parameter READ_ADD = 3'b011;
parameter READ_DATA = 3'b100;

input clk,rst_n,SS_n,MOSI,tx_valid;
input [7:0] tx_data;
output reg MISO ,rx_valid;
output reg [9:0] rx_data;
(* fsm_encoding = "sequential" *)
reg [2:0] cs,ns;
reg read_state;
reg [4:0] counter;

// STATE MEMORY
always@(posedge clk)begin
    if(!rst_n) begin cs <= IDLE; read_state <= 0; end
    else begin
        cs <= ns;

        end
    end

// NEXT STATE
always@(cs,SS_n,MOSI)begin
    case(cs)
        IDLE : begin
            if(SS_n) ns=IDLE;
            else ns=CHK_CMD;
        end
        CHK_CMD : begin
            if(SS_n) ns=IDLE;
            else begin
                if(!MOSI) ns=WRITE;
                else if(!read_state) ns=READ_ADD;
                else ns=READ_DATA;
            end
        end
        WRITE : begin
            if(SS_n) ns=IDLE;
            else ns=WRITE;
        end
        READ_ADD : begin
            if(SS_n) ns=IDLE;
            else ns=READ_ADD;
        end
        READ_DATA : begin
            if(SS_n) ns=IDLE;
            else ns=READ_DATA;
        end
        default : ns = IDLE;
    endcase
end

// OUTPUT LOGIC
always@(posedge clk)begin
    if (!rst_n) begin
        rx_valid <= 0;
        counter <= 0;
        MISO <= 0;
        rx_data <=0;
    end else begin
        case(cs)
            IDLE : begin rx_valid<=0; end
            CHK_CMD : counter<=0;
            WRITE : begin
                if(counter ==10)rx_valid <=1;
                else begin counter <=counter+1; rx_data[9-counter] <= MOSI; end
            end
            READ_ADD : begin
                if(counter ==10)begin rx_valid <=1; read_state <= 1;end
                else begin counter <=counter+1; rx_data[9-counter] <= MOSI; end
            end
            READ_DATA : begin
                if(!tx_valid)begin
                    if(counter == 10)begin counter <=0;rx_valid <=1; end
                    else if(counter < 10 && !rx_valid)begin counter <=counter+1; rx_data[9-counter] <= MOSI; end
                end
                else if(tx_valid)begin
                    rx_valid <=0;
                    if(counter == 8)begin counter <=0;read_state <= 0; end
                    else if(counter < 8)begin counter <=counter+1; MISO <= tx_data[7-counter];end
                end
            end
        endcase
    end
end
endmodule
```

8) Interface

```
interface spi_IF(input clk);

    logic        MOSI;
    logic        rst_n;
    logic        SS_n;
    logic        tx_valid;
    logic [7:0]   tx_data;
    logic [9:0]   rx_data;
    logic        rx_valid;
    logic        MISO;
    logic [9:0]   rx_data_ref;
    logic        rx_valid_ref;
    logic        MISO_ref;

    modport DUT (input  MOSI, clk, rst_n, SS_n, tx_valid, tx_data,output rx_data, rx_valid, MISO);
    modport SVA (input  MOSI, clk, rst_n, SS_n, tx_valid, tx_data,rx_data, rx_valid, MISO);
endinterface
```

9) Top Module

```
import uvm_pkg::*;
import spi_test_pkg::*;
`include "uvm_macros.svh"
module top;
    bit clk;
    initial forever #1 clk=~clk;

    spi_IF spi_vif(clk);

    SLAVE DUT(spi_vif);

    SPI_SLAVE GOLDEN_MODEL (
        .clk      (spi_vif.clk),
        .rst_n    (spi_vif.rst_n),
        .SS_n     (spi_vif.SS_n),
        .rx_data   (spi_vif.rx_data_ref),
        .rx_valid  (spi_vif.rx_valid_ref),
        .tx_data   (spi_vif.tx_data),
        .tx_valid  (spi_vif.tx_valid),
        .MOSI      (spi_vif.MOSI),
        .MISO      (spi_vif.MISO_ref)
    );

    bind DUT spi_SVA sva_inst(spi_vif);

    initial begin
        uvm_config_db#(virtual spi_IF) :: set(null,"uvm_test_top","V_IF",spi_vif);
        run_test("spi_test");
    end
endmodule
```

10) Test Package

```
package spi_test_pkg;
    import uvm_pkg::*;
    import spi_env_pkg::*;
    import spi_main_sequence_pkg::*;
    import spi_reset_sequence_pkg::*;
    import config_obj_pkg::*;
    `include "uvm_macros.svh"

    class spi_test extends uvm_test;
        `uvm_component_utils(spi_test)
        spi_env env;
        config_obj cfg;
        spi_main_sequence main_seq;
        spi_reset_sequence rst_seq;
        virtual spi_IF v_if;

        function new(string name = "spi_test", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = spi_env ::type_id::create("env", this);
            cfg = config_obj::type_id::create("cfg");
            main_seq = spi_main_sequence::type_id::create("main_seq");
            rst_seq = spi_reset_sequence::type_id::create("rst_seq");
            if(!uvm_config_db#(virtual spi_IF)::get(this, "", "V_IF", cfg.spi_config_vif))
                `uvm_fatal("build_phase", "error")
            uvm_config_db#(config_obj)::set(this, "*", "CFG", cfg);
            cfg.is_active = UVM_ACTIVE;
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            phase.raise_objection(this);
            `uvm_info("run_phase", "Reset Sequence Start", UVM_MEDIUM);
            rst_seq.start(env.agt.sqr);
            `uvm_info("run_phase", "Reset Sequence End", UVM_MEDIUM);
            `uvm_info("run_phase", "Main Sequence Start", UVM_MEDIUM);
            main_seq.start(env.agt.sqr);
            `uvm_info("run_phase", "Main Sequence End", UVM_MEDIUM);
            phase.drop_objection(this);
        endtask

    endclass
endpackage
```

11) Env Package



```
package spi_env_pkg;
import uvm_pkg::*;
import spi_agent_pkg::*;
import spi_scoreboard_pkg::*;
import spi_cvr_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_env extends uvm_env;
    `uvm_component_utils(spi_env)
    spi_agent agt;
    spi_scoreboard sb;
    spi_coveragecollector cvr;
    spi_seq_item seq_item;
    function new(string name = "spi_env",uvm_component parent = null);
        super.new(name,parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt = spi_agent::type_id::create("agent",this);
        sb = spi_scoreboard::type_id::create("sb",this);
        cvr = spi_coveragecollector::type_id::create("cvr",this);
    endfunction
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.agt_ap.connect(sb.sb_export);
        agt.agt_ap.connect(cvr.cvr_export);
    endfunction
endclass
endpackage
```

12) Agent Package

```
package spi_agent_pkg;
import uvm_pkg::*;
import spi_driver_pkg::*;
import spi_monitor_pkg::*;
import config_obj_pkg::*;
import spi_sequencer_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_agent extends uvm_agent;
    `uvm_component_utils(spi_agent)
    config_obj cfg;
    spi_driver drv;
    spi_monitor mon;
    spi_sequencer sqr;
    uvm_analysis_port #(spi_seq_item) agt_ap;

    function new(string name = "spi_agent", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(config_obj)::get(this, "", "CFG", cfg))
            `uvm_fatal("build phase", "error")
        if(cfg.is_active == UVM_ACTIVE)begin
            sqr = spi_sequencer::type_id::create("sqr", this);
            drv = spi_driver::type_id::create("drv", this);
        end
        mon = spi_monitor::type_id::create("mon", this);
        agt_ap = new("agt_ap", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        if(cfg.is_active == UVM_ACTIVE)begin
            drv.spi_driver_vif = cfg.spi_config_vif;
            drv.seq_item_port.connect(sqr.seq_item_export);
        end
        mon.v_if = cfg.spi_config_vif;
        mon.mon_ap.connect(agt_ap);
    endfunction

endclass
endpackage
```

13) Reset Sequence

```
package spi_reset_sequence_pkg;
import uvm_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_reset_sequence extends uvm_sequence #(spi_seq_item);
  `uvm_object_utils(spi_reset_sequence)
  spi_seq_item seq;
  function new(string name = "spi_reset_sequence");
    super.new(name);
  endfunction
  task body;
    seq = spi_seq_item ::type_id::create("seq");
    start_item(seq);
    seq.rst_n = 0;
    finish_item(seq);
  endtask
endclass
endpackage
```

14) Main Sequence

```
package spi_main_sequence_pkg;
import uvm_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_main_sequence extends uvm_sequence #(spi_seq_item);
  `uvm_object_utils(spi_main_sequence)
  spi_seq_item seq;
  function new(string name = "spi_main_sequence");
    super.new(name);
  endfunction
  task body;
    seq = spi_seq_item ::type_id::create("seq");
    repeat(50000)begin
      start_item(seq);
      assert(seq.randomize());
      finish_item(seq);
    end
  endtask
endclass
endpackage
```

15) Sequencer Package



```
package spi_sequencer_pkg;
import uvm_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_sequencer extends uvm_sequencer #(spi_seq_item);
    `uvm_component_utils(spi_sequencer)
    function new(string name = "spi_sequencer", uvm_component parent = null);
        super.new(name, parent);
    endfunction
endclass
endpackage
```

16) Sequence Item Package

```
package spi_sequence_item_pkg;
import uvm_pkg::*;
localparam IDLE      = 3'b000;
localparam WRITE     = 3'b001;
localparam CHK_CMD   = 3'b010;
localparam READ_ADD  = 3'b011;
localparam READ_DATA = 3'b100;
`include "uvm_macros.svh"
class spi_seq_item extends uvm_sequence_item;
    `uvm_object_utils(spi_seq_item)
    rand logic rst_n;
    logic MOSI,tx_valid,SS_n;
    logic rx_valid,MISO,rx_valid_ref,MISO_ref;
    rand logic [7:0] tx_data;
    logic [7:0] tx_data_old;
    logic [9:0] rx_data,rx_data_ref;
    rand bit [10:0] MOSI_bits;
    bit [10:0] MOSI_bits_old;
    static integer counter = 0;
    static integer bit_index = 10;
    static bit new_frame_needed = 1;
    static logic received_address = 1;

    constraint rst_constraint{
        rst_n dist{0:=2,1:=98};
    }
    constraint mosi_constraint{
        MOSI_bits[10:8] inside {3'b000, 3'b001, 3'b110, 3'b111};
    }

    function new(string name = "spi_seq_item");
        super.new(name);
    endfunction

    function void post_randomize();
        if(!rst_n)begin
            counter = 0;
            SS_n=1;
            new_frame_needed = 1;
            bit_index = 10;
            tx_valid = 0;
            received_address = 1;
        end
        else begin
            counter++;
            if (new_frame_needed) begin
                MOSI_bits_old = MOSI_bits;
                tx_data_old = tx_data;
                bit_index = 10;
                new_frame_needed = 0;
                if(MOSI_bits_old[9:8]==2'b10 && received_address)begin received_address = 0;end
                else if(MOSI_bits_old[9:8]==2'b10 && !received_address)begin MOSI_bits_old[9:8] = 2'b11; received_address = 1;end
                else if(MOSI_bits_old[9:8]==2'b11 && !received_address)begin received_address = 1;end
                else if (MOSI_bits_old[9:8]==2'b11 && received_address) begin MOSI_bits_old[9:8] = 2'b10; received_address = 0; end
            end

            if(counter>=2)begin
                MOSI = MOSI_bits_old[bit_index];
                bit_index--;
            end
            if (bit_index < 0) begin
                bit_index = 10;
            end
            if (MOSI_bits_old[9:8] == 2'b11) begin
                if(counter == 14)tx_valid = 1;
                else if (counter == 23)begin
                    SS_n = 1;
                    counter = 0;
                    new_frame_needed = 1;
                    tx_valid = 0;
                end
                else begin
                    SS_n = 0;
                end
            end
            end
            else begin
                tx_valid = 0;
                if (counter == 14)begin
                    SS_n = 1;
                    counter = 0;
                    new_frame_needed = 1;
                end
                else begin
                    SS_n = 0;
                end
            end
        end
    endfunction
```

```
function string convert2string();
    return $sformatf(
        "Cycle=%0d | rst_n=%0b | SS_n=%0b | MOSI=%0b | MISO=%0b | tx_valid=%0b | rx_valid=%0b | tx_data=0x%0h | rx_data=0x%0h | MOSI_bits=%b",
        counter, rst_n, SS_n, MOSI, MISO, tx_valid, rx_valid, tx_data_old, rx_data, MOSI_bits
    );
endfunction
```

```
function string convert2string_stimulus();
    return $sformatf(
        "Cycle=%0d | rst_n=%0b | SS_n=%0b | tx_valid=%0b | rx_valid=%0b | MOSI_bits=%b | tx_data=0x%0h | rx_data=0x%0h",
        counter, rst_n, SS_n, tx_valid, rx_valid, MOSI_bits, tx_data_old, rx_data
    );
endfunction
```

```
endclass
endpackage
```


17) Driver Package

```
package spi_driver_pkg;
import uvm_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_driver extends uvm_driver#(spi_seq_item);
    `uvm_component_utils(spi_driver)
    virtual spi_IF spi_driver_vif;
    spi_seq_item seq_item;

    function new(string name = "spi_driver",uvm_component parent = null);
        super.new(name,parent);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            seq_item = spi_seq_item :: type_id:: create("seq_item");
            seq_item_port.get_next_item(seq_item);
            spi_driver_vif.MOSI      = seq_item.MOSI;
            spi_driver_vif.rst_n     = seq_item.rst_n;
            spi_driver_vif.SS_n      = seq_item.SS_n;
            spi_driver_vif.tx_valid  = seq_item.tx_valid;
            spi_driver_vif.tx_data   = seq_item.tx_data_old;
            seq_item.rx_data         = spi_driver_vif.rx_data;
            @(negedge spi_driver_vif.clk);
            seq_item_port.item_done();
            `uvm_info("run_phase",seq_item.convert2string_stimulus(),UVM_HIGH);
        end
    endtask

endclass
endpackage
```

18) Monitor Package

```
package spi_monitor_pkg;
import uvm_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_monitor extends uvm_monitor;
    `uvm_component_utils(spi_monitor)
    virtual spi_IF v_if;
    spi_seq_item seq_item;
    uvm_analysis_port #(spi_seq_item) mon_ap;

    function new(string name = "spi_monitor",uvm_component parent = null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon_ap = new("mon_ap",this);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            seq_item = spi_seq_item::type_id::create("seq_item");
            seq_item.MOSI      = v_if.MOSI;
            seq_item.rst_n     = v_if.rst_n;
            seq_item.SS_n      = v_if.SS_n;
            seq_item.tx_valid  = v_if.tx_valid;
            seq_item.tx_data   = v_if.tx_data;
            seq_item.rx_data   = v_if.rx_data;
            seq_item.rx_valid  = v_if.rx_valid;
            seq_item.MISO      = v_if.MISO;
            seq_item.rx_data_ref = v_if.rx_data_ref;
            seq_item.rx_valid_ref = v_if.rx_valid_ref;
            seq_item.MISO_ref  = v_if.MISO_ref;
            @(negedge v_if.clk);
            mon_ap.write(seq_item);
        end
    endtask
endclass
endpackage
```

19) Scoreboard Package

```
package spi_scoreboard_pkg;
import uvm_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"

class spi_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(spi_scoreboard)

    spi_seq_item seq_item;
    uvm_analysis_export #(spi_seq_item) sb_export;
    uvm_tlm_analysis_fifo #(spi_seq_item) sb_fifo;

    logic [5:0] out_ref;
    logic [15:0] leds_ref;

    int correct_counter = 0;
    int error_counter = 0;

    function new(string name = "spi_scoreboard", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export", this);
        sb_fifo = new("sb_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            sb_fifo.get(seq_item);
            if ((seq_item.rx_data != seq_item.rx_data_ref || seq_item.rx_valid != seq_item.rx_valid_ref || seq_item.MISO != seq_item.MISO_ref))
                error_counter++;
            else
                correct_counter++;
        end
    endtask

    function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        $display("Final Counters for spi %d,%d", correct_counter, error_counter);
    endfunction

endclass
endpackage
```

20) Coverage Package

```
package spi_cvr_pkg;
import uvm_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_coveragecollector extends uvm_component;
    `uvm_component_utils(spi_coveragecollector)
    spi_seq_item seq_item;
    uvm_analysis_export #(spi_seq_item) cvr_export;
    uvm_tlm_analysis_fifo #(spi_seq_item) cvr_fifo;
    covergroup cvr_grp;
        rx_cp : coverpoint seq_item.rx_data[9:8]{
            bins write_addr  = {2'b00};
            bins write_data  = {2'b01};
            bins read_addr   = {2'b10};
            bins read_data   = {2'b11};
            bins transitions[] = (2'b00, 2'b01, 2'b10, 2'b11 => 2'b00, 2'b01, 2'b10, 2'b11);
            // Due to grey encoding 2 bits can't change at same time so these transitions will never happens
            ignore_bins illegal_transitions_2[] = (2'b10=> 2'b01);
            ignore_bins illegal_transitions_3[] = (2'b01=> 2'b10);
            ignore_bins illegal_transitions_4[] = (2'b00 => 2'b11);
        }

        SSn_cp : coverpoint seq_item.SSn_n {
            bins normal_transaction  = (1 => 0[*13] => 1);
            bins extended_transaction = (1 => 0[*22] => 1);
        }

        MOSI_cp : coverpoint seq_item.MOSI{
            bins write_addr[] = (0 => 0 => 0);
            bins write_data[] = (0 => 0 => 1);
            bins read_addr[]  = (1 => 1 => 0);
            bins read_data[]  = (1 => 1 => 1);
        }

        x_SS_MOSI : cross SSn_cp, MOSI_cp{
            ignore_bins extended_000 = binsof(MOSI_cp.write_addr) && binsof(SSn_cp.extended_transaction);
            ignore_bins extended_001 = binsof(MOSI_cp.write_data) && binsof(SSn_cp.extended_transaction);
            ignore_bins extended_110 = binsof(MOSI_cp.read_addr)  && binsof(SSn_cp.extended_transaction);
            ignore_bins normal_111  = binsof(MOSI_cp.read_data)  && binsof(SSn_cp.normal_transaction);
        }
    endgroup

    function new(string name = "spi_coveragecollector",uvm_component parent = null);
        super.new(name,parent);
        cvr_grp = new();
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        cvr_export = new("cvr_export",this);
        cvr_fifo = new("cvr_fifo",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        cvr_export.connect(cvr_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cvr_fifo.get(seq_item);
            cvr_grp.sample();
        end
    endtask
endclass
endpackage
```

21) Config Object

```
package config_obj_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
class config_obj extends uvm_object;
    `uvm_object_utils(config_obj)
    virtual spi_IF spi_config_vif;
    uvm_active_passive_enum is_active;

    function new(string name = "config_obj");
        super.new(name);
    endfunction
endclass
endpackage
```

22) Do File / Src File

```
vlib work
vlog -f src_files.list +cover +define+SIM -covercells
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover
coverage save spi.ucdb -onexit
add wave /top/spi_vif/*
run -all
coverage exclude -src SPI_slave.sv -line 22 -code b
coverage exclude -src SPI_slave.sv -line 72 -code b
quit -sim
vcover report spi.ucdb -details -annotate -all -output spi.txt
```

```
spi_if.sv
spi_sva.sv
SPI_golden.v
SPI_slave.sv
config_obj.sv
spi_sequence_item.sv
spi_reset_sequence.sv
spi_main_sequence.sv
spi_sequencer.sv
spi_driver.sv
spi_monitor.sv
spi_agent.sv
spi_sb.sv
spi_cvr.sv
spi_env.sv
spi_test.sv
top.sv
```

23) FSM Coverage

FSM Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
FSM States	5	5	0	100.00%
FSM Transitions	8	8	0	100.00%

=====FSM Details=====

FSM Coverage for instance /top/DUT --

FSM_ID: cs

Current State Object : cs

State Value MapInfo :

Line	State Name	Value
----	-----	-----
23	IDLE	0
29	CHK_CMD	1
55	READ_DATA	4
49	READ_ADD	3
43	WRITE	2

Covered States :

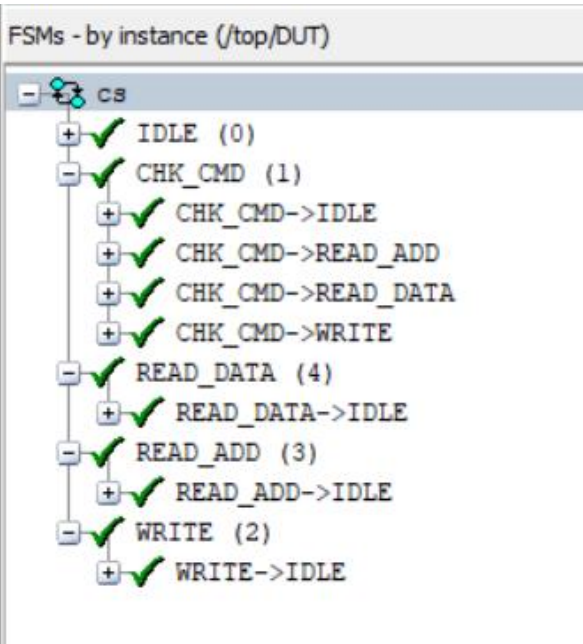
State	Hit_count
----	-----
IDLE	3904
CHK_CMD	3839
READ_DATA	1382
READ_ADD	2316
WRITE	3716

Covered Transitions :

Line	Trans_ID	Hit_count	Transition
----	-----	-----	-----
27	0	3839	IDLE -> CHK_CMD
39	1	699	CHK_CMD -> READ_DATA
37	2	1169	CHK_CMD -> READ_ADD
34	3	1876	CHK_CMD -> WRITE
31	4	95	CHK_CMD -> IDLE
57	5	699	READ_DATA -> IDLE
51	6	1169	READ_ADD -> IDLE
45	7	1875	WRITE -> IDLE

Summary

	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
FSM States	5	5	0	100.00%
FSM Transitions	8	8	0	100.00%



24) Branch Coverage

- After excluding the Case branch as it need default to be able to get hit 100%

```
Branch Coverage:

  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Branches              28      28      0    100.00%

=====Branch Details=====

Branch Coverage for instance /top/DUT

  line      Item              Count    Source
  ----      -
  File SPI_slave.sv

-----IF Branch-----
  13              15157    Count coming in to IF
  13          1          1055          if (~spi_vif.rst_n) begin
  16          1          14102          else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
  24              7705    Count coming in to IF
  24          1          3865          if (spi_vif.SS_n)
  26          1          3840          else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
  30              5811    Count coming in to IF
  30          1           95          if (spi_vif.SS_n)
  32          1          5716          else begin

Branch totals: 2 hits of 2 branches = 100.00%
```

25) Statement Coverage



Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	39	39	0	100.00%

=====Statement Details=====

Statement Coverage for instance /top/DUT --

Line	Item	Count	Source
----	----	-----	-----
File SPI_slave.sv			
1			module SLAVE (spi_IF.DUT spi_vif);
2			localparam IDLE = 3'b000;
3			localparam CHK_CMD = 3'b001; // change chk_cmd value from 2 to 1
4			localparam WRITE = 3'b010;
5			localparam READ_ADD = 3'b011;
6			localparam READ_DATA = 3'b100;
7			
8			reg [3:0] counter;
9			reg received_address;
10			reg [2:0] cs, ns;
11			
12	1	15157	always @(posedge spi_vif.clk) begin

26) Toggle Coverage

```
Toggle Coverage:

  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Toggles               22      22       0  100.00%

=====Toggle Details=====

Toggle Coverage for instance /top/DUT --

      Node      1H->0L    0L->1H  "Coverage"
      -----
      counter[3-0]      1        1    100.00
      cs[2-0]           1        1    100.00
      ns[2-0]           1        1    100.00
      received_address   1        1    100.00

Total Node Count    =      11
Toggled Node Count  =      11
Untoggled Node Count =       0

Toggle Coverage     =    100.00% (22 of 22 bins)
```

27) Functional Coverage

Covergroups										
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Corr	
CVP cvr_grp::ssn_cp		100.00%	100	100.00...		✓				
CVP cvr_grp::MOSI_cp		100.00%	100	100.00...		✓				
CROSS cvr_grp::x_SS_MOSI		100.00%	100	100.00...		✓				
INST \spi_cvr_pkg::spi_coveragecollector::cvr_grp		100.00%	100	100.00...		✓			0	
CVP rx_cp		100.00%	100	100.00...		✓				
B ignore_bin illegal_transitions_2[2=>1]		0	-	-		✓				
B ignore_bin illegal_transitions_3[1=>2]		0	-	-		✓				
B ignore_bin illegal_transitions_4[0=>3]		0	-	-		✓				
B bin write_addr		14127	1	100.00...		✓				
B bin write_data		10727	1	100.00...		✓				
B bin read_addr		13121	1	100.00...		✓				
B bin read_data		12025	1	100.00...		✓				
B bin transitions[3=>3]		11023	1	100.00...		✓				
B bin transitions[3=>2]		438	1	100.00...		✓				
B bin transitions[3=>1]		332	1	100.00...		✓				
B bin transitions[3=>0]		232	1	100.00...		✓				
B bin transitions[2=>3]		539	1	100.00...		✓				
B bin transitions[2=>2]		11854	1	100.00...		✓				
B bin transitions[2=>0]		728	1	100.00...		✓				
B bin transitions[1=>3]		463	1	100.00...		✓				
B bin transitions[1=>1]		9774	1	100.00...		✓				
B bin transitions[1=>0]		490	1	100.00...		✓				
B bin transitions[0=>2]		829	1	100.00...		✓				
B bin transitions[0=>1]		621	1	100.00...		✓				
B bin transitions[0=>0]		12676	1	100.00...		✓				
CVP SSn_cp		100.00%	100	100.00...		✓				
B bin normal_transaction		2457	1	100.00...		✓				
B bin extended_transaction		470	1	100.00...		✓				
CVP MOSI_cp		100.00%	100	100.00...		✓				
B bin write_addr[0=>0=>0]		8830	1	100.00...		✓				
B bin write_data[0=>0=>1]		5992	1	100.00...		✓				
B bin read_addr[1=>1=>0]		6218	1	100.00...		✓				
B bin read_data[1=>1=>1]		8464	1	100.00...		✓				
CROSS x_SS_MOSI		100.00%	100	100.00...		✓				
B bin <extended_transaction,read_data[1=>1=>...]		67	1	100.00...		✓				
B bin <normal_transaction,read_addr[1=>1=>0]...		341	1	100.00...		✓				
B bin <normal_transaction,write_data[0=>0=>1]...		230	1	100.00...		✓				
B bin <normal_transaction,write_addr[0=>0=>0]...		397	1	100.00...		✓				
B ignore_bin normal_111		233	-	-		✓				
B ignore_bin extended_110		74	-	-		✓				
B ignore_bin extended_001		50	-	-		✓				
B ignore_bin extended_000		46	-	-		✓				

28) Assertions

/spi_main_sequence_pkg::spi_main_sequence::body/#ublk#224849143...	Immediate	SVA	on	0	1	-	-	-	-	0	off	assert (/randomize(...))	✓
/top/DUT/a_1	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge spi_vif.clk) disa...	✓
/top/DUT/a_2	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge spi_vif.clk) disa...	✓
/top/DUT/a_3	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge spi_vif.clk) disa...	✓
/top/DUT/a_4	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge spi_vif.clk) disa...	✓
/top/DUT/a_5	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge spi_vif.clk) disa...	✓
/top/DUT/ava_inst/assert1	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge spi_vif.clk) (~sp...	✓
/top/DUT/ava_inst/assert2	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge spi_vif.clk) disa...	✓

29) Assertions Coverage

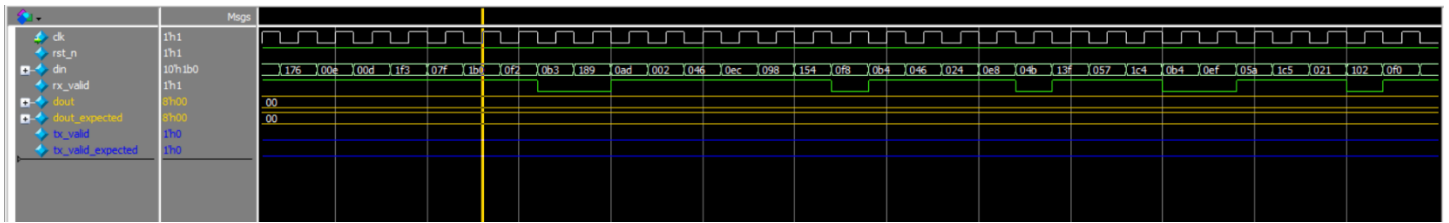
Cover Directives														
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/top/DUT/c_1	SVA	✓	Off	3839	1	Unli...	1	100%	██████████	✓	0	0	0 ns	0
/top/DUT/c_2	SVA	✓	Off	3744	1	Unli...	1	100%	██████████	✓	0	0	0 ns	0
/top/DUT/c_3	SVA	✓	Off	1448	1	Unli...	1	100%	██████████	✓	0	0	0 ns	0
/top/DUT/c_4	SVA	✓	Off	896	1	Unli...	1	100%	██████████	✓	0	0	0 ns	0
/top/DUT/c_5	SVA	✓	Off	455	1	Unli...	1	100%	██████████	✓	0	0	0 ns	0
/top/DUT/ava_inst/cover1	SVA	✓	Off	1055	1	Unli...	1	100%	██████████	✓	0	0	0 ns	0
/top/DUT/ava_inst/cover2	SVA	✓	Off	48324	1	Unli...	1	100%	██████████	✓	0	0	0 ns	0

Part 2: UVM Environment for Single-Port RAM

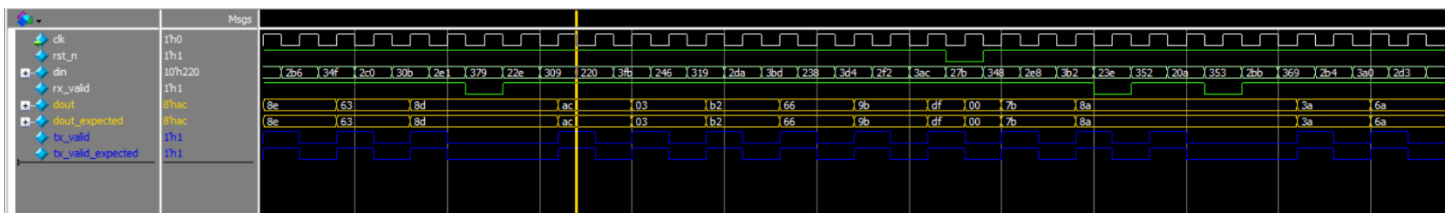
Interface:

Name	Type	Size	Description
din	Input	10 bits	Data Input
clk		1 bit	Clock
rst		1 bit	Active low synchronous reset
rx_valid		1 bit	If HIGH: accept din[7:0] to save the write/read address internally or write a memory word depending on the most significant 2 bits din[9:8]
dout	Output	8 bits	Data Output
tx_valid		1 bit	Whenever the command is memory read the tx_valid should be HIGH

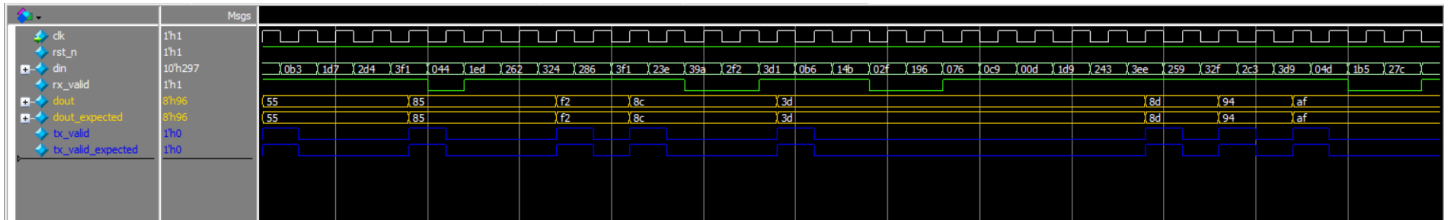
Write Addr or Write Data Wave (Write Only)



Read Addr or Read Data Wave (Read Only)



Write Read Wave



2) Transcript Output

```
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test ram_test...
# UVM_INFO ram_test.sv(43) @ 0: uvm_test_top [run_phase] reset asserted
# *****
# * Questa UVM Transaction Recording Turned ON. *
# * recording_detail has been set. *
# * To turn off, set 'recording_detail' to off: *
# * uvm_config_db#(int) ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# *****
# UVM_INFO ram_test.sv(45) @ 2: uvm_test_top [run_phase] reset deasserted
# UVM_INFO ram_test.sv(48) @ 2: uvm_test_top [run_phase] write only stimulus generation asserted
# UVM_INFO ram_test.sv(50) @ 34002: uvm_test_top [run_phase] write only stimulus generation deasserted
# UVM_INFO ram_test.sv(53) @ 34002: uvm_test_top [run_phase] read only stimulus generation asserted
# UVM_INFO ram_test.sv(55) @ 68002: uvm_test_top [run_phase] read only stimulus generation deasserted
# UVM_INFO ram_test.sv(58) @ 68002: uvm_test_top [run_phase] write read stimulus generation asserted
# UVM_INFO ram_test.sv(60) @ 102002: uvm_test_top [run_phase] write read stimulus generation deasserted
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 102002: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# Final Counters for spi 51001, 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 12
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [run_phase] 8
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 102002 ns Iteration: 61 Instance: /top
# 1
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

3) UVM Factory

```
uvm printfactory
#
#### Factory Configuration (*)
#
#   No instance or type overrides are registered with this factory
#
# All types registered with the factory: 52 total
# (types without type names will not be printed)
#
#   Type Name
#   -----
#   questa_uvm_recorder
#   ram_agent
#   ram_config_obj
#   ram_coverage
#   ram_driver
#   ram_env
#   ram_monitor
#   ram_read_only_seq
#   ram_rst_seq
#   ram_scoreboard
#   ram_seq_item
#   ram_sequencer
#   ram_test
#   ram_write_only_seq
#   ram_write_read_seq
# (*) Types with no associated type name will be printed as <unknown>
#
####
#
```

4) Design Code (After Bugs Correction)

```
module RAM (din,clk,rst_n,rx_valid,dout,tx_valid);
input      [9:0] din;
input      clk, rst_n, rx_valid;
output reg [7:0] dout;
output reg      tx_valid;

reg [7:0] MEM [255:0];

reg [7:0] Rd_Addr, Wr_Addr;

always @(posedge clk) begin
    if (~rst_n) begin
        dout <= 0;
        tx_valid <= 0;
        Rd_Addr <= 0;
        Wr_Addr <= 0;
    end
    else begin //add begin , end
        if (rx_valid) begin
            case (din[9:8])
                2'b00 : Wr_Addr <= din[7:0];
                2'b01 : MEM[Wr_Addr] <= din[7:0];
                2'b10 : Rd_Addr <= din[7:0];
                2'b11 : dout <= MEM[Rd_Addr]; //should read from rd_addr not wr_addr
                default : dout <= 0;
            endcase
        end
        tx_valid <= (din[9] && din[8] && rx_valid)? 1'b1 : 1'b0;
    end
end

endmodule
```


5) SVA Module

```
module SVA(ram_if.SVA r);
  //reset
  property preset;
    @(posedge r.clk) !r.rst_n | => (r.dout == 0 && r.tx_valid == 0);
  endproperty
  areset : assert property (preset)
    else $error("Assertion areset failed! at time (%0t)" , $time);
  creset : cover property(preset);

  // low tx_valid
  property ptx_valid_l;
    @(posedge r.clk) disable iff(!r.rst_n) (r.din[9:8] == 2'b00 || r.din[9:8] == 2'b01 || r.din[9:8] == 2'b10) && (r.rx_valid == 1'b1) | => r.tx_valid == 0 ;
  endproperty
  atx_valid_l : assert property (ptx_valid_l)
    else $error("Assertion ptx_valid_l failed! at time (%0t)" , $time);
  ctx_valid_l : cover property(ptx_valid_l);

  // high tx_valid
  property ptx_valid_h;
    @(posedge r.clk) disable iff(!r.rst_n) (r.din[9:8] == 2'b11) && (r.rx_valid == 1'b1) | => $rose(r.tx_valid) ##1 $fell(r.tx_valid) ;
  endproperty
  atx_valid_h : assert property (ptx_valid_h)
    else $error("Assertion ptx_valid_h failed! at time (%0t)" , $time);
  ctx_valid_h : cover property(ptx_valid_h);

  //wr_addr => wr_data
  property pwr_addr_data;
    @(posedge r.clk) disable iff(!r.rst_n) (r.din[9:8] == 2'b00) | -> ##[1:$] (r.din[9:8] == 2'b01) ;
  endproperty
  awr_addr_data : assert property (pwr_addr_data)
    else $error("Assertion pwr_addr_data failed! at time (%0t)" , $time);
  cwr_addr_data : cover property(pwr_addr_data);

  //rd_addr => rd_data
  property prd_addr_data;
    @(posedge r.clk) disable iff(!r.rst_n) (r.din[9:8] == 2'b10) | -> ##[1:$] (r.din[9:8] == 2'b11) ;
  endproperty
  ardr_addr_data : assert property (prd_addr_data)
    else $error("Assertion prd_addr_data failed! at time (%0t)" , $time);
  crdr_addr_data : cover property(prd_addr_data);
endmodule
```

6) Golden Model



```
module RAM_gmodel(clk,rst_n,rx_valid,din,tx_valid,dout);

parameter MEM_WIDTH = 8;
parameter MEM_DEPTH = 256;
parameter ADDR_SIZE = 8;

input clk,rst_n,rx_valid;
input [9:0] din;
output reg tx_valid;
output reg [7:0] dout;
reg [ADDR_SIZE-1:0] wr_addr,rd_addr;

reg [MEM_WIDTH-1:0] mem [MEM_DEPTH-1:0];

always@(posedge clk)begin
    if(!rst_n) begin dout<=0; tx_valid<=0; wr_addr<=0; rd_addr<=0; end
    else if(rx_valid) begin
        case(din[9:8])
            2'b00 : begin wr_addr <= din[7:0]; tx_valid<=0; end
            2'b01 : begin mem[wr_addr] <= din[7:0]; tx_valid<=0; end
            2'b10 : begin rd_addr <= din[7:0]; tx_valid<=0; end
            2'b11 : begin dout <= mem[rd_addr]; tx_valid<= 1; end
            default : begin tx_valid <=0; dout<= 0;end
        endcase
    end
    else begin
        tx_valid <= 0;
    end
end
endmodule
```

7) Interface

```
interface ram_if(input bit clk);
parameter MEM_DEPTH = 256;
parameter ADDR_SIZE = 8;
logic [9:0] din;
logic      rst_n, rx_valid;
logic [7:0] dout, dout_expected;
logic      tx_valid, tx_valid_expected;

modport SVA(
input din, clk, rst_n, rx_valid, dout, tx_valid);
endinterface
```

8) Top Module

```
import ram_test_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
module top();
bit clk;
initial begin
    clk = 0;
    forever begin
        #1 clk = ~clk;
    end
end

ram_if r_if(clk);

RAM DUT(r_if.din, r_if.clk, r_if.rst_n, r_if.rx_valid, r_if.dout, r_if.tx_valid);

RAM_gmodel r_gmodel(r_if.clk, r_if.rst_n, r_if.rx_valid, r_if.din, r_if.tx_valid_expected, r_if.dout_expected);

bind RAM SVA sva(r_if.SVA);

initial begin
    uvm_config_db#(virtual ram_if)::set(null, "uvm_test_top", "ram_if", r_if);
    run_test("ram_test");
end

endmodule
```

9) Test Package

```
package ram_test_pkg;
import ram_env_pkg::*;
import ram_rst_seq_pkg::*;
import ram_write_only_seq_pkg::*;
import ram_read_only_seq_pkg::*;
import ram_write_read_seq_pkg::*;
import ram_config_obj_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_test extends uvm_test;
  `uvm_component_utils(ram_test)
  ram_env env;
  ram_config_obj ram_cfg;
  ram_rst_seq rst_seq;
  ram_write_only_seq wr_only_seq;
  ram_read_only_seq rd_only_seq;
  ram_write_read_seq wr_rd_seq;
  virtual ram_if ram_vif;

  function new(string name = "ram_test", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = ram_env::type_id::create("env",this);
    ram_cfg = ram_config_obj::type_id::create("ram_cfg");
    rst_seq = ram_rst_seq::type_id::create("rst_seq");
    wr_only_seq = ram_write_only_seq::type_id::create("wr_only_seq");
    rd_only_seq = ram_read_only_seq::type_id::create("rd_only_seq");
    wr_rd_seq = ram_write_read_seq::type_id::create("wr_rd_seq");
    ram_cfg.is_active = UVM_ACTIVE;
    if(!uvm_config_db #(virtual ram_if)::get(this, "", "ram_if", ram_cfg.ram_config_vif))
      `uvm_fatal("build_phase", "Test - unable to get the virtual interface of the ram from the uvm_config_db");
    uvm_config_db #(ram_config_obj)::set(this, "*", "CFG", ram_cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
    //rst_sequence
    `uvm_info("run_phase", "reset asserted", UVM_LOW)
    rst_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "reset deasserted", UVM_LOW)

    //write_only_sequence
    `uvm_info("run_phase", "write only stimulus generation asserted", UVM_LOW)
    wr_only_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "write only stimulus generation deasserted", UVM_LOW)

    //read_only_sequence
    `uvm_info("run_phase", "read only stimulus generation asserted", UVM_LOW)
    rd_only_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "read only stimulus generation deasserted", UVM_LOW)

    //write_read_sequence
    `uvm_info("run_phase", "write read stimulus generation asserted", UVM_LOW)
    wr_rd_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "write read stimulus generation deasserted", UVM_LOW)

    phase.drop_objection(this);
  endtask
endclass

endpackage
```

10) Env Package

```
package ram_env_pkg;
import ram_agent_pkg::*;
import ram_coverage_pkg::*;
import ram_scoreboard_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_env extends uvm_env;
  `uvm_component_utils(ram_env)
  ram_agent agt;
  ram_scoreboard sb;
  ram_coverage cov;

  function new(string name = "ram_env" , uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = ram_agent::type_id::create("agt",this);
    cov = ram_coverage::type_id::create("cov",this);
    sb = ram_scoreboard::type_id::create("sb",this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    agt.agt_ap.connect(cov.cov_export);
    agt.agt_ap.connect(sb.sb_export);
  endfunction

endclass

endpackage
```

11) Agent Package

```
package ram_agent_pkg;
import ram_driver_pkg::*;
import ram_sequencer_pkg::*;
import ram_monitor_pkg::*;
import ram_config_obj_pkg::*;
import ram_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_agent extends uvm_agent;
  `uvm_component_utils(ram_agent)
  ram_driver driver;
  ram_sequencer sqr;
  ram_monitor mon;
  ram_config_obj ram_cfg;
  uvm_analysis_port #(ram_seq_item) agt_ap;

  function new(string name = "ram_agent" , uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon = ram_monitor::type_id::create("mon",this);
    if(!uvm_config_db#(ram_config_obj)::get(this, "", "CFG", ram_cfg))
      `uvm_fatal("build_phase", "agent - unable to get the config object");
    if(ram_cfg.is_active == UVM_ACTIVE)begin
      driver = ram_driver::type_id::create("driver",this);
      sqr = ram_sequencer::type_id::create("sqr",this);
    end
    agt_ap = new("agt_ap", this);

  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    if(ram_cfg.is_active == UVM_ACTIVE)begin
      driver.ram_driver_vif = ram_cfg.ram_config_vif;
      driver.seq_item_port.connect(sqr.seq_item_export);
    end
    mon.ram_monitor_vif = ram_cfg.ram_config_vif;
    mon.mon_ap.connect(agt_ap);
  endfunction

endclass

endpackage
```

12) Rst Sequence

```
package ram_rst_seq_pkg;
import shared_pkg::*;
import ram_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_rst_seq extends uvm_sequence #(ram_seq_item);
`uvm_object_utils(ram_rst_seq)
    ram_seq_item seq_item;

    function new(string name = "ram_rst_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = ram_seq_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 0;
        seq_item.din = 0;
        seq_item.rx_valid = 0;
        finish_item(seq_item);
    endtask
endclass
endpackage
```

13) Write Only Sequence

```
package ram_write_only_seq_pkg;
import shared_pkg::*;
import ram_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_write_only_seq extends uvm_sequence #(ram_seq_item);
`uvm_object_utils(ram_write_only_seq)
    ram_seq_item seq_item;

    function new(string name = "ram_write_only_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = ram_seq_item::type_id::create("seq_item");
        repeat(17000) begin
            start_item(seq_item);
            assert(seq_item.randomize() with {din[9:8] inside {[0:1]}});
            finish_item(seq_item);
        end
    endtask
endclass
endpackage
```

14) Read Only Sequence

```
package ram_read_only_seq_pkg;
import shared_pkg::*;
import ram_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_read_only_seq extends uvm_sequence #(ram_seq_item);
  `uvm_object_utils(ram_read_only_seq)
  ram_seq_item seq_item;

  function new(string name = "ram_read_only_seq");
    super.new(name);
  endfunction

  task body;
    seq_item = ram_seq_item::type_id::create("seq_item");
    seq_item.wr_c.constraint_mode(0); //closing to prevent conflict
    seq_item.rd_wr.constraint_mode(0);
    repeat(17000) begin
      start_item(seq_item);
      assert(seq_item.randomize() with {din[9:8] inside {[2:3]}});
      finish_item(seq_item);
    end
  endtask
endclass
endpackage
```

15) Read Write Sequence

```
package ram_write_read_seq_pkg;
import ram_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_write_read_seq extends uvm_sequence #(ram_seq_item);
  `uvm_object_utils(ram_write_read_seq)
  ram_seq_item seq_item;

  function new(string name = "ram_write_read_seq");
    super.new(name);
  endfunction

  task body;
    seq_item = ram_seq_item::type_id::create("seq_item");
    seq_item.rd_wr.constraint_mode(1); //reopen after closing
    seq_item.rd_c.constraint_mode(0);
    repeat(17000) begin
      start_item(seq_item);
      assert(seq_item.randomize());
      finish_item(seq_item);
    end
  endtask
endclass
endpackage
```


16) Sequencer Package



```
package ram_sequencer_pkg;
import ram_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_sequencer extends uvm_sequencer #(ram_seq_item);
  `uvm_component_utils(ram_sequencer)

  function new(string name = "ram_sequencer", uvm_component parent = null);
    super.new(name,parent);
  endfunction

endclass
endpackage
```

17) Sequence Item Package

```
package ram_seq_item_pkg;
import shared_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_seq_item extends uvm_sequence_item;
`uvm_object_utils(ram_seq_item)

    rand logic [9:0] din;
    rand logic clk, rst_n, rx_valid;
    logic [7:0] dout, dout_expected;
    logic tx_valid, tx_valid_expected;
    op_t old_op;

    function new(string name = "ram_seq_item");
        super.new(name);
    endfunction

    constraint const_w {
        rst_n dist {0:=2 , 1:=98};
        rx_valid dist {0:=20 , 1:=80};
    }

    constraint wr_c {
        if(old_op == WRITE_ADDRESS)
        {
            din[9:8] inside {WRITE_ADDRESS, WRITE_DATA};
        }
    }

    constraint rd_c {
        if(old_op == READ_ADDRESS)
        {
            din[9:8] == READ_DATA;
        }
        else if(old_op == READ_DATA)
        {
            din[9:8] == READ_ADDRESS;
        }
    }

    constraint rd_wr {
        if(old_op == WRITE_ADDRESS)
        {
            din[9:8] inside {WRITE_ADDRESS, WRITE_DATA};
        }
        if(old_op == READ_ADDRESS)
        {
            din[9:8] == READ_DATA;
        }
        if(old_op == WRITE_DATA)
        {
            din[9:8] dist {READ_ADDRESS:/60, WRITE_ADDRESS:/40};
        }
        else if(old_op == READ_DATA)
        {
            din[9:8] dist {WRITE_ADDRESS:/60, READ_ADDRESS:/40};
        }
    }

    function void post_randomize;
        old_op = op_t'(din[9:8]);
    endfunction

    function string convert2string();
        return $sformatf("%s rst_n = 0b%0b, din = 0b%0b, rx_valid = 0b%0b, dout = 0b%0b, tx_valid = 0b%0b",
            super.convert2string(), rst_n, din, rx_valid, dout, tx_valid);
    endfunction

    function string convert2string_stimulus();
        return $sformatf("rst_n = 0b%0b, din = 0b%0b, rx_valid = 0b%0b",
            rst_n, din, rx_valid);
    endfunction
endclass
endpackage
```

18) Driver Package



```
package ram_driver_pkg;
import ram_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_driver extends uvm_driver #(ram_seq_item);
  `uvm_component_utils(ram_driver)
  virtual ram_if ram_driver_vif;
  ram_seq_item seq_item;

  function new(string name = "ram_driver", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      seq_item = ram_seq_item::type_id::create("seq_item");
      seq_item_port.get_next_item(seq_item);
      ram_driver_vif.rst_n = seq_item.rst_n;
      ram_driver_vif.din = seq_item.din;
      ram_driver_vif.rx_valid = seq_item.rx_valid;
      @(negedge ram_driver_vif.clk);
      seq_item_port.item_done();
      `uvm_info("run_phase", seq_item.convert2string_stimulus(), UVM_HIGH);
    end
  endtask
endclass

endpackage
```

19) Monitor Package

```
package ram_monitor_pkg;
import ram_seq_item_pkg::*;
import shared_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_monitor extends uvm_monitor;
    `uvm_component_utils(ram_monitor)
    virtual ram_if ram_monitor_vif;
    ram_seq_item seq_item;
    uvm_analysis_port #(ram_seq_item) mon_ap;

    function new(string name = "ram_monitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon_ap = new("mon_ap", this);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            seq_item = ram_seq_item::type_id::create("seq_item");
            @(negedge ram_monitor_vif.clk);
            seq_item.rst_n = ram_monitor_vif.rst_n;
            seq_item.din = ram_monitor_vif.din;
            seq_item.rx_valid = ram_monitor_vif.rx_valid;
            seq_item.dout = ram_monitor_vif.dout;
            seq_item.dout_expected = ram_monitor_vif.dout_expected;
            seq_item.tx_valid = ram_monitor_vif.tx_valid;
            seq_item.tx_valid_expected = ram_monitor_vif.tx_valid_expected;
            mon_ap.write(seq_item);
            `uvm_info("run_phase", seq_item.convert2string(), UVM_HIGH)
        end
    endtask
endclass

endpackage
```

20) Scoreboard Package

```
package ram_scoreboard_pkg;
import ram_seq_item_pkg::*;
import shared_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(ram_scoreboard)
  uvm_analysis_export #(ram_seq_item) sb_export;
  uvm_tlm_analysis_fifo #(ram_seq_item) sb_fifo;
  ram_seq_item seq_item;
  integer error_count = 0;
  integer correct_count = 0;

  function new(string name = "ram_scoreboard", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sb_export = new("sb_export", this);
    sb_fifo = new("sb_fifo", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    sb_export.connect(sb_fifo.analysis_export);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      sb_fifo.get(seq_item);
      if (seq_item.dout != seq_item.dout_expected || seq_item.tx_valid != seq_item.tx_valid_expected) begin
        `uvm_error("run phase", $sformatf("comparison failed: %s ",seq_item.convert2string()));
        error_count++;
      end
      else begin
        `uvm_info("run_phase", $sformatf("comparison passed: %s", seq_item.convert2string()), UVM_HIGH);
        correct_count++;
      end
    end
  endtask

  function void report_phase(uvm_phase phase);
    super.report_phase(phase);
    $display("Final Counters for spi %d,%d", correct_count, error_count);
  endfunction

endclass
endpackage
```

21) Coverage Package

```
package ram_coverage_pkg;
import ram_seq_item_pkg::*;
import uvm_pkg::*;
import shared_pkg::*;
`include "uvm_macros.svh"
class ram_coverage extends uvm_component;
`uvm_component_utils(ram_coverage)
    uvm_analysis_export #(ram_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(ram_seq_item) cov_fifo;
    ram_seq_item seq_item;
    logic [9:0] din;
    logic tx_valid, rx_valid;

    covergroup Covram;

        din_cp : coverpoint din[9:8] {
            bins write_addr = {2'b00};
            bins write_data = {2'b01};
            bins read_addr = {2'b10};
            bins read_data = {2'b11};
        }

        wr_data_after_wr_addr : coverpoint din[9:8] iff(rx_valid){
            bins wr_data_wr_addr = (2'b00 => 2'b01);
        }

        rd_data_after_rd_addr : coverpoint din[9:8] iff(rx_valid){
            bins rd_data_rd_addr = (2'b10 => 2'b11);
        }

        cp_rd_wr : coverpoint din[9:8] iff(rx_valid){
            bins rd_wr = (2'b00 => 2'b01 => 2'b10 => 2'b11);
        }

        cross_din_rx : cross din_cp, rx_valid {
            option.cross_auto_bin_max =0;
            bins din_rx = binsof(din_cp) && binsof(rx_valid) intersect {1'b1};
        }

        cross_read_tx : cross din_cp, tx_valid {
            option.cross_auto_bin_max =0;
            bins rd_tx = binsof(din_cp.read_data) && binsof(tx_valid) intersect {1'b1};
        }

    endgroup

    function new(string name = "ram_coverage", uvm_component parent = null);
        super.new(name,parent);
        Covram = new();
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        cov_export = new("cov_export", this);
        cov_fifo = new("cov_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        cov_export.connect(cov_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cov_fifo.get(seq_item);
            din = seq_item.din;
            rx_valid = seq_item.rx_valid;
            tx_valid = seq_item.tx_valid;

            if (seq_item.rst_n) begin
                Covram.sample();
            end
        end
    endtask
endclass
endpackage
```

22) Config Obj Package



```
package ram_config_obj_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_config_obj extends uvm_object;
    `uvm_object_utils(ram_config_obj)
    virtual ram_if ram_config_vif;
    uvm_active_passive_enum is_active;

    function new(string name = "ram_config_obj");
        super.new(name);
    endfunction

endclass
endpackage
```

23) Shared Package



```
package shared_pkg;
typedef enum {WRITE_ADDRESS, WRITE_DATA, READ_ADDRESS, READ_DATA } op_t;
endpackage
```

24) Do File / Src File



```
vlib work  
vlog -f src_files.list +cover -covercells  
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover  
coverage save ram.ucdb -onexit  
add wave /top/r_if/*  
run -all  
coverage exclude -src RAM.v -line 25 -code s  
coverage exclude -src RAM.v -line 25 -code b  
quit -sim  
vcover report ram.ucdb -details -annotate -all -output ram.txt
```



```
shared_pkg.sv  
SVA.sv  
ram_if.sv  
RAM.v  
RAM_golden_model.v  
ram_seq_item.sv  
ram_rst_seq.sv  
ram_write_only_seq.sv  
ram_read_only_seq.sv  
ram_write_read_seq.sv  
ram_sequencer.sv  
ram_driver.sv  
ram_monitor.sv  
ram_config_obj.sv  
ram_agent.sv  
ram_scoreboard.sv  
ram_coverage.sv  
ram_env.sv  
ram_test.sv  
top_module.sv
```


25) Branch Coverage (After Default Exclude)

```

=====
Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Branches              8       8       0    100.00%

=====Branch Details=====

Branch Coverage for instance /top/DUT

  Line    Item              Count    Source
  ----    -
  File RAM.v

-----IF Branch-----
  12              50999    Count coming in to IF
  12      1              1023      if (~rst_n) begin
  18      1              49976      else begin //add begin , end

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
  19              49976    Count coming in to IF
  19      1              39880      if (rx_valid) begin
  10096    All False Count

Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----
  20              39880    Count coming in to CASE
  21      1              14170      2'b00 : Wr_Addr <= din[7:0];
  22      1              7127      2'b01 : MEM[Wr_Addr] <= din[7:0];
  23      1              9307      2'b10 : Rd_Addr <= din[7:0];
  24      1              9276      2'b11 : dout <= MEM[Rd_Addr]; //should read from rd_addr not wr_addr

Branch totals: 4 hits of 4 branches = 100.00%
```

26) Statement Coverage (After Default Exclude)

```
Statement Coverage:
  Enabled Coverage      Bins  Hits  Misses Coverage
  -----
  Statements            10    10     0   100.00%

=====Statement Details=====

Statement Coverage for instance /top/DUT --

  Line   Item          Count  Source
  ----   -
File RAM.v
  1                               module RAM (din,clk,rst_n,rx_valid,dout,tx_valid);
  2                               input    [9:0] din;
  3                               input      clk, rst_n, rx_valid;
  4                               output reg [7:0] dout;
  5                               output reg    tx_valid;
  6
  7                               reg [7:0] MEM [255:0];
  8
  9                               reg [7:0] Rd_Addr, Wr_Addr;
 10
 11          1                50999  always @(posedge clk) begin
 12                               if (~rst_n) begin
 13          1                1023      dout <= 0;
 14          1                1023      tx_valid <= 0;
 15          1                1023      Rd_Addr <= 0;
 16          1                1023      Wr_Addr <= 0;
 17                               end
 18                               else begin //add begin , end
 19                               if (rx_valid) begin
 20                               case (din[9:8])
 21          1                14170      2'b00 : Wr_Addr <= din[7:0];
 22          1                7127      2'b01 : MEM[Wr_Addr] <= din[7:0];
 23          1                9307      2'b10 : Rd_Addr <= din[7:0];
 24          1                9276      2'b11 : dout <= MEM[Rd_Addr]; //should read from rd_addr not wr_addr
 25                               default : dout <= 0;
 26                               endcase
 27                               end
 28          1                49976      tx_valid <= (din[9] && din[8] && rx_valid)? 1'b1 : 1'b0;
```

27) Toggle Coverage



Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	76	76	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /top/DUT --

Node	1H->0L	0L->1H	"Coverage"

Rd_Addr[7-0]	1	1	100.00
Wr_Addr[7-0]	1	1	100.00
clk	1	1	100.00
din[0-9]	1	1	100.00
dout[7-0]	1	1	100.00
rst_n	1	1	100.00
rx_valid	1	1	100.00
tx_valid	1	1	100.00

Total Node Count = 38
Toggled Node Count = 38
Untoggled Node Count = 0

Toggle Coverage = 100.00% (76 of 76 bins)

28) Functional Coverage

Covergroups									
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/ram_coverage_pkg/ram_coverage		100.00%							
TYPE Covram		100.00%	100	100.00...		✓			auto(0)
CVP Covram::din_cp		100.00%	100	100.00...		✓			
CVP Covram::wr_data_after_wr_addr		100.00%	100	100.00...		✓			
CVP Covram::rd_data_after_rd_addr		100.00%	100	100.00...		✓			
CVP Covram::cp_rd_wr		100.00%	100	100.00...		✓			
CVP Covram::tx_valid		100.00%	100	100.00...		✓			
CVP Covram::rx_valid		100.00%	100	100.00...		✓			
CROSS Covram::cross_din_rx		100.00%	100	100.00...		✓			
CROSS Covram::cross_read_tx		100.00%	100	100.00...		✓			
INST \ram_coverage_pkg::ram_coverage::Covram		100.00%	100	100.00...		✓			0
CVP din_cp		100.00%	100	100.00...		✓			
bin write_addr		17821	1	100.00...		✓			
bin write_data		8882	1	100.00...		✓			
bin read_addr		11629	1	100.00...		✓			
bin read_data		11646	1	100.00...		✓			
CVP wr_data_after_wr_addr		100.00%	100	100.00...		✓			
bin wr_data_wr_addr		6409	1	100.00...		✓			
CVP rd_data_after_rd_addr		100.00%	100	100.00...		✓			
bin rd_data_rd_addr		7575	1	100.00...		✓			
CVP cp_rd_wr		100.00%	100	100.00...		✓			
bin rd_wr		912	1	100.00...		✓			
CVP tx_valid		100.00%	100	100.00...		✓			
bin auto[0]		40702	1	100.00...		✓			
bin auto[1]		9276	1	100.00...		✓			
CVP rx_valid		100.00%	100	100.00...		✓			
bin auto[0]		10098	1	100.00...		✓			
bin auto[1]		39880	1	100.00...		✓			
CROSS cross_din_rx		100.00%	100	100.00...		✓			
bin din_rx		39880	1	100.00...		✓			
CROSS cross_read_tx		100.00%	100	100.00...		✓			
bin rd_tx		9276	1	100.00...		✓			

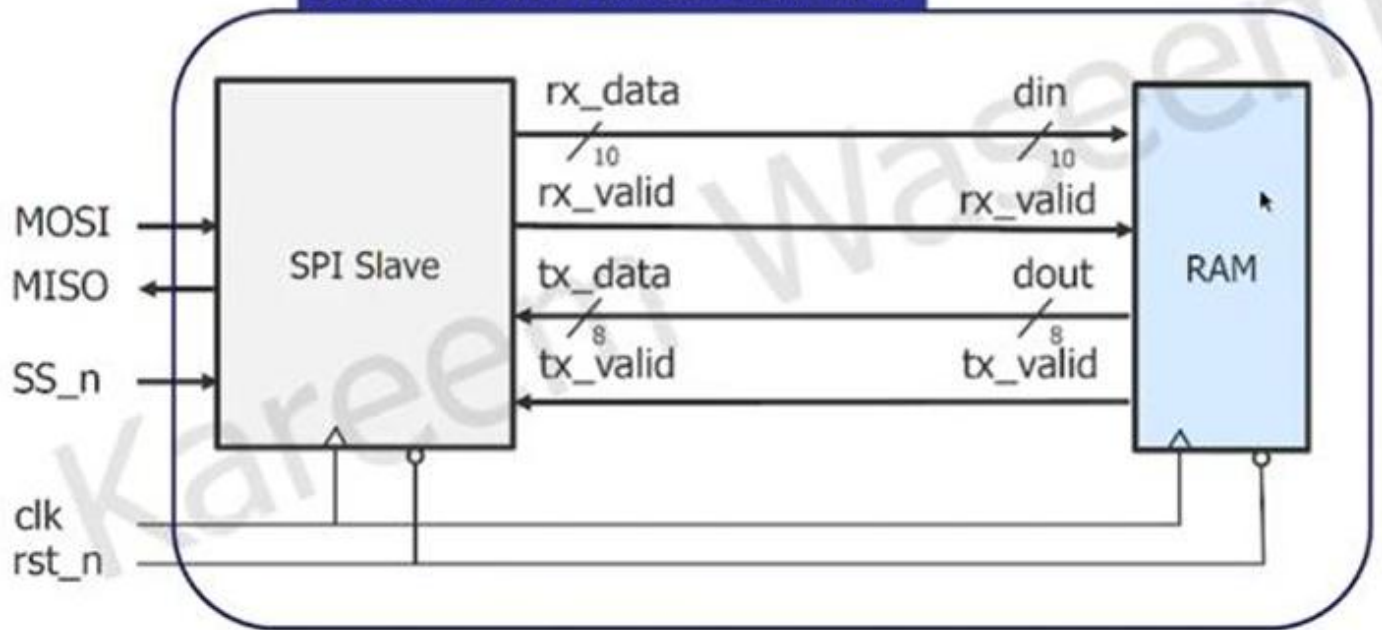
29) Assertions

▲ /ram_write_read_seq_pkg::ram_write_read_seq::body/#ublk#1634866...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
▲ /ram_read_only_seq_pkg::ram_read_only_seq::body/#ublk#244094295...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
▲ /ram_write_only_seq_pkg::ram_write_only_seq::body/#ublk#22576251...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
+▲ /top/DUT/sva/areset	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge r.clk) (~r.rst_n)...	✓
+▲ /top/DUT/sva/atx_valid_l	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge r.clk) disable iff...	✓
+▲ /top/DUT/sva/atx_valid_h	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge r.clk) disable iff...	✓
+▲ /top/DUT/sva/awr_addr_data	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge r.clk) disable iff...	✓
+▲ /top/DUT/sva/ard_addr_data	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge r.clk) disable iff...	✓

30) Assertions Coverage

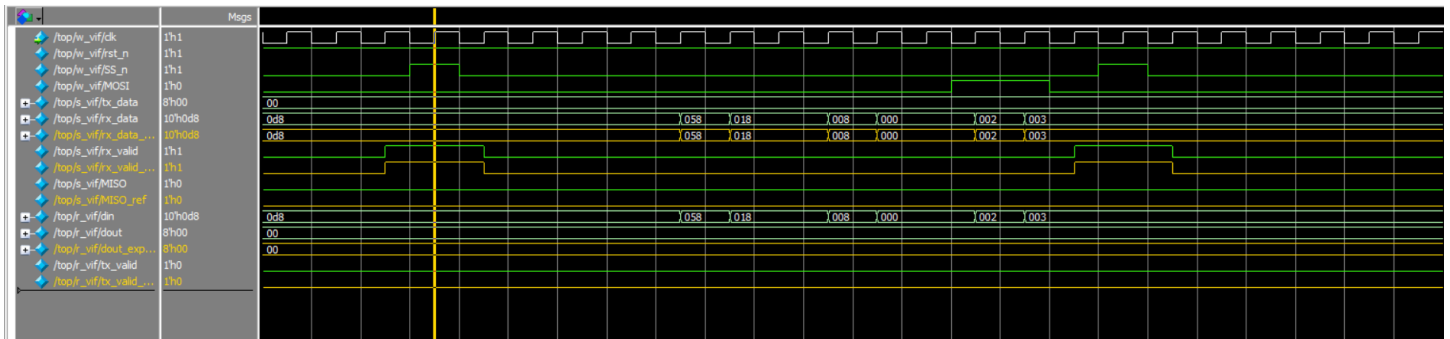
Cover Directives														
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
▲ /top/DUT/sva/creset	SVA	✓	Off	1023	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
▲ /top/DUT/sva/ctx_valid_l	SVA	✓	Off	30028	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
▲ /top/DUT/sva/ctx_valid_h	SVA	✓	Off	8896	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
▲ /top/DUT/sva/cwr_addr_data	SVA	✓	Off	17044	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
▲ /top/DUT/sva/crd_addr_data	SVA	✓	Off	11405	1	Unli...	1	100%	✓	✓	0	0	0 ns	0

SPI SLAVE WITH SINGLE PORT RAM

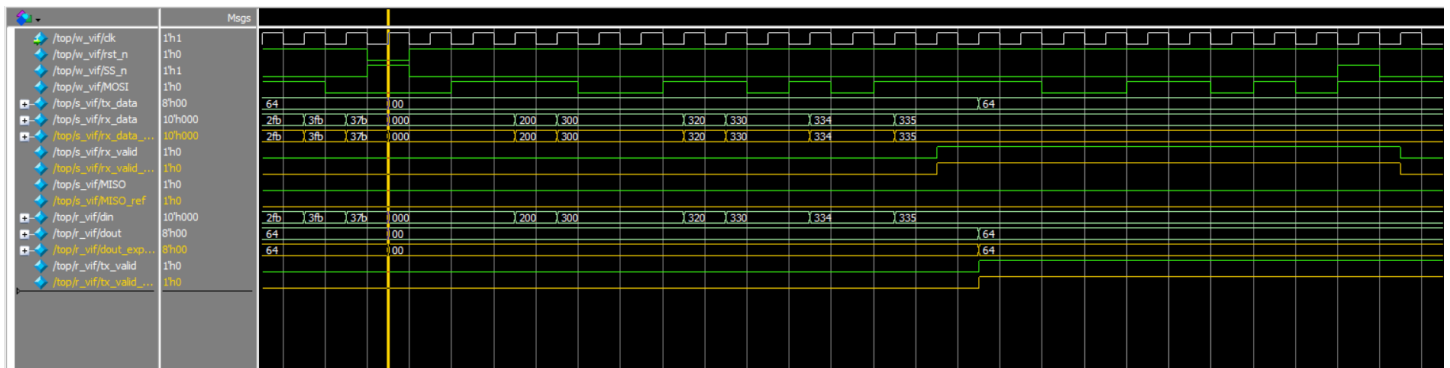


1) Waveform Snippets

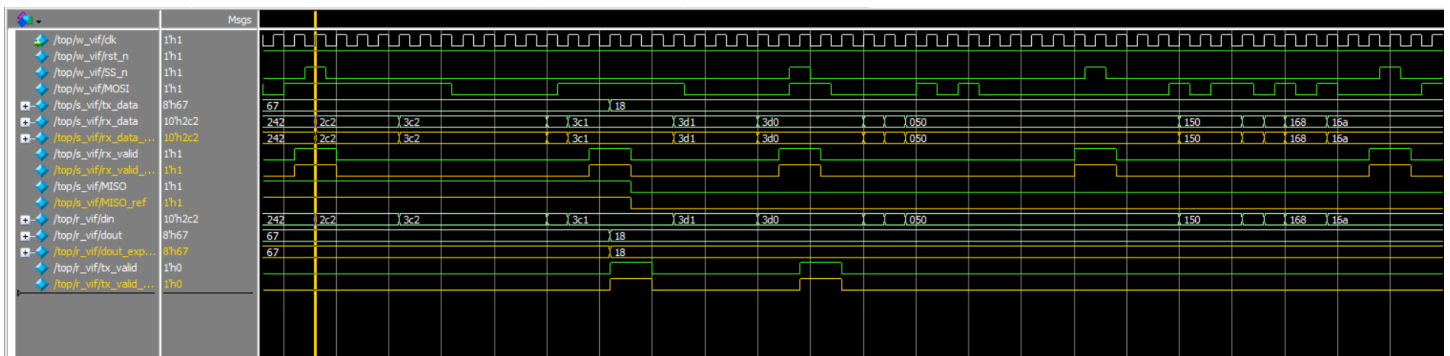
Write Only Sequence



Read Only Sequence



Write Read Sequence



2) Transcript Output

```
#
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNIST] Running test wrapper_test...
# UVM_INFO wrapper_test.sv(72) @ 0: uvm_test_top [run_phase] reset asserted
# *****
# * Questa UVM Transaction Recording Turned ON. *
# * recording_detail has been set. *
# * To turn off, set 'recording_detail' to off: *
# * uvm_config_db#(int) ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# *****
# UVM_INFO wrapper_test.sv(74) @ 2: uvm_test_top [run_phase] reset deasserted
# UVM_INFO wrapper_test.sv(77) @ 2: uvm_test_top [run_phase] write only stimulus generation asserted
# UVM_INFO wrapper_test.sv(79) @ 34002: uvm_test_top [run_phase] write only stimulus generation deasserted
# UVM_INFO wrapper_test.sv(82) @ 34002: uvm_test_top [run_phase] read only stimulus generation asserted
# UVM_INFO wrapper_test.sv(84) @ 68002: uvm_test_top [run_phase] read only stimulus generation deasserted
# UVM_INFO wrapper_test.sv(87) @ 68002: uvm_test_top [run_phase] write read stimulus generation asserted
# UVM_INFO wrapper_test.sv(89) @ 102002: uvm_test_top [run_phase] write read stimulus generation deasserted
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 102002: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# Final Counters for ram 51001, 0
# Final Counters for spi 51001, 0
# Final Counters for wrapper 51001, 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 12
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNIST] 1
# [TEST_DONE] 1
# [run_phase] 8
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 102002 ns Iteration: 61 Instance: /top
# 1
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```


3) UVM Factory

```
# All types registered with the factory: 72 total
# (types without type names will not be printed)
#
#   Type Name
#   -----
#   config_obj
#   questa_uvm_recorder
#   ram_agent
#   ram_config_obj
#   ram_coverage
#   ram_driver
#   ram_env
#   ram_monitor
#   ram_scoreboard
#   ram_seq_item
#   ram_sequencer
#   spi_agent
#   spi_coveragecollector
#   spi_driver
#   spi_env
#   spi_monitor
#   spi_scoreboard
#   spi_seq_item
#   spi_sequencer
#   wrapper_agent
#   wrapper_config_obj
#   wrapper_coveragecollector
#   wrapper_driver
#   wrapper_env
#   wrapper_monitor
#   wrapper_read_only_seq
#   wrapper_rst_seq
#   wrapper_scoreboard
#   wrapper_seq_item
#   wrapper_sequencer
#   wrapper_test
#   wrapper_write_only_seq
#   wrapper_write_read_seq
# (*) Types with no associated type name will be printed as <unknown>
#
```

4) Design Code



```
module WRAPPER (MOSI,MISO,SS_n,clk,rst_n);

input  MOSI, SS_n, clk, rst_n;
output MISO;

wire [9:0] rx_data_din;
wire      rx_valid;
wire      tx_valid;
wire [7:0] tx_data_dout;

RAM  RAM_instance  (rx_data_din,clk,rst_n,rx_valid,tx_data_dout,tx_valid);
SLAVE SLAVE_instance (MOSI,MISO,SS_n,clk,rst_n,rx_data_din,rx_valid,tx_data_dout,tx_valid);

endmodule
```

5) SVA Model

```
module wrapper_SVA(wrapper_IF.SVA w_vif);

    property rst_prp;
        @(posedge w_vif.clk) !w_vif.rst_n | => (!w_vif.MISO);
    endproperty
    assert1 : assert property(rst_prp);
    cover1 : cover property(rst_prp);

    sequence write_addr_sequence;
        w_vif.SS_n ##2 !w_vif.MOSI ##1 !w_vif.MOSI ##1 !w_vif.MOSI;
    endsequence
    sequence write_data_sequence;
        w_vif.SS_n ##2 !w_vif.MOSI ##1 !w_vif.MOSI ##1 w_vif.MOSI;
    endsequence
    sequence read_addr_sequence;
        w_vif.SS_n ##2 w_vif.MOSI ##1 w_vif.MOSI ##1 !w_vif.MOSI;
    endsequence

    property miso_stable;
        @(posedge w_vif.clk) disable iff(!w_vif.rst_n)
            (write_addr_sequence or write_data_sequence or read_addr_sequence) | -> $stable(w_vif.MISO);
    endproperty
    assert2 : assert property(miso_stable);
    cover2 : cover property(miso_stable);

endmodule
```

6) Golden Model

```
module SPI_WRAPPER_GOLDEN(  
    input MOSI,  
    output MISO,  
    input SS_n,  
    input clk,  
    input rst_n  
);  
  
    // Internal signals  
    wire [9:0] rx_data;  
    wire rx_valid;  
    wire [7:0] tx_data;  
    wire tx_valid;  
  
    // SPI_SLAVE instantiation  
    SPI_SLAVE spi_slave_inst (  
        .clk(clk),  
        .rst_n(rst_n),  
        .SS_n(SS_n),  
        .rx_data(rx_data),  
        .rx_valid(rx_valid),  
        .tx_data(tx_data),  
        .tx_valid(tx_valid),  
        .MOSI(MOSI),  
        .MISO(MISO)  
    );  
  
    // RAM instantiation  
    RAM ram_inst (  
        .clk(clk),  
        .rst_n(rst_n),  
        .rx_valid(rx_valid),  
        .din(rx_data),  
        .tx_valid(tx_valid),  
        .dout(tx_data)  
    );  
  
endmodule
```

7) Interface

```
interface wrapper_IF(input clk);

    logic MOSI, SS_n, rst_n, MISO, MISO_ref;

    modport DUT (input MOSI, clk, rst_n, SS_n, output MISO);
    modport SVA (input MOSI, clk, rst_n, SS_n, MISO);
endinterface
```

8) Top Module

```
import uvm_pkg::*;
import wrapper_test_pkg::*;
`include "uvm_macros.svh"
module top;
    bit clk;
    initial forever #1 clk=~clk;

    wrapper_IF w_vif(clk);
    spi_IF s_vif(clk);
    ram_IF r_vif(clk);

    // Wrapper
    WRAPPER W_DUT(w_vif.MOSI,w_vif.MISO,w_vif.SS_n,w_vif.clk,w_vif.rst_n);
    SPI_WRAPPER_GOLDEN W_GOLDEN(w_vif.MOSI,w_vif.MISO_ref,w_vif.SS_n,w_vif.clk,w_vif.rst_n);

    // SPI SLAVE
    assign s_vif.rx_data = W_DUT.rx_data_din;
    assign s_vif.rst_n = W_DUT.rst_n;
    assign s_vif.rx_valid = W_DUT.rx_valid;
    assign s_vif.tx_data = W_DUT.tx_data_dout;
    assign s_vif.tx_valid = W_DUT.tx_valid;
    assign s_vif.rx_data_ref = W_GOLDEN.rx_data;
    assign s_vif.rx_valid_ref = W_GOLDEN.rx_valid;
    assign s_vif.MISO_ref = W_GOLDEN.MISO;
    assign s_vif.MISO = W_DUT.MISO;
    assign s_vif.SS_n = W_DUT.SS_n;
    assign s_vif.MOSI = W_DUT.MOSI;

    // RAM
    assign r_vif.din = W_DUT.rx_data_din;
    assign r_vif.rst_n = W_DUT.rst_n;
    assign r_vif.rx_valid = W_DUT.rx_valid;
    assign r_vif.dout = W_DUT.tx_data_dout;
    assign r_vif.dout_expected = W_GOLDEN.tx_data;
    assign r_vif.tx_valid = W_DUT.tx_valid;
    assign r_vif.tx_valid_expected = W_GOLDEN.tx_valid;

    bind W_DUT wrapper_SVA wrapper_inst(w_vif.SVA);
    bind W_DUT.SLAVE_instance spi_SVA spi_inst(s_vif.SVA);
    bind W_DUT.RAM_instance ram_SVA ram_inst(r_vif.SVA);

    initial begin
        uvm_config_db#(virtual wrapper_IF) :: set(null,"uvm_test_top","W_IF",w_vif);
        uvm_config_db#(virtual spi_IF) :: set(null,"uvm_test_top","S_IF",s_vif);
        uvm_config_db#(virtual ram_IF) :: set(null,"uvm_test_top","R_IF",r_vif);
        run_test("wrapper_test");
    end
endmodule
```

9) Test Package

```
package wrapper_test_pkg;
import uvm_pkg::*;
import wrapper_env_pkg::*;
import wrapper_rst_sequence_pkg::*;
import wrapper_write_only_sequence_pkg::*;
import wrapper_read_only_sequence_pkg::*;
import wrapper_write_read_sequence_pkg::*;
import config_obj_pkg::*;
import ram_config_obj_pkg::*;
import wrapper_config_obj_pkg::*;
import spi_env_pkg::*;
import ram_env_pkg::*;
import wrapper_env_pkg::*;
`include "uvm_macros.svh"

class wrapper_test extends uvm_test;
    `uvm_component_utils(wrapper_test)
    spi_env s_env;
    ram_env r_env;
    wrapper_env w_env;
    config_obj s_cfg;
    ram_config_obj r_cfg;
    wrapper_config_obj w_cfg;
    wrapper_rst_seq rst_seq;
    wrapper_write_only_seq wr_only_seq;
    wrapper_read_only_seq rd_only_seq;
    wrapper_write_read_seq wr_rd_seq;
    virtual spi_IF s_if;
    virtual ram_IF r_if;
    virtual wrapper_IF w_if;

    function new(string name = "wrapper_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        s_env = spi_env ::type_id::create("s_env", this);
        r_env = ram_env ::type_id::create("r_env", this);
        w_env = wrapper_env ::type_id::create("w_env", this);

        s_cfg = config_obj::type_id::create("s_cfg");
        r_cfg = ram_config_obj::type_id::create("r_cfg");
        w_cfg = wrapper_config_obj::type_id::create("w_cfg");

        rst_seq = wrapper_rst_seq::type_id::create("rst_seq");
        wr_only_seq = wrapper_write_only_seq::type_id::create("wr_only_seq");
        rd_only_seq = wrapper_read_only_seq::type_id::create("rd_only_seq");
        wr_rd_seq = wrapper_write_read_seq::type_id::create("wr_rd_seq");

        if(!uvm_config_db#(virtual spi_IF)::get(this, "", "S_IF", s_cfg.spi_config_vif))
            `uvm_fatal("build phase", "error")
        if(!uvm_config_db#(virtual ram_IF)::get(this, "", "R_IF", r_cfg.ram_config_vif))
            `uvm_fatal("build phase", "error")
        if(!uvm_config_db#(virtual wrapper_IF)::get(this, "", "W_IF", w_cfg.wrapper_config_vif))
            `uvm_fatal("build phase", "error")

        s_cfg.is_active = UVM_PASSIVE;
        r_cfg.is_active = UVM_PASSIVE;
        w_cfg.is_active = UVM_ACTIVE;

        uvm_config_db#(config_obj)::set(this, "", "S_CFG", s_cfg);
        uvm_config_db#(ram_config_obj)::set(this, "", "R_CFG", r_cfg);
        uvm_config_db#(wrapper_config_obj)::set(this, "", "W_CFG", w_cfg);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        phase.raise_objection(this);

        //rst_sequence
        `uvm_info("run_phase", "reset asserted", UVM_LOW)
        rst_seq.start(w_env.agt.sqr);
        `uvm_info("run_phase", "reset deasserted", UVM_LOW)

        //write_only_sequence
        `uvm_info("run_phase", "write only stimulus generation asserted", UVM_LOW)
        wr_only_seq.start(w_env.agt.sqr);
        `uvm_info("run_phase", "write only stimulus generation deasserted", UVM_LOW)

        //read_only_sequence
        `uvm_info("run_phase", "read only stimulus generation asserted", UVM_LOW)
        rd_only_seq.start(w_env.agt.sqr);
        `uvm_info("run_phase", "read only stimulus generation deasserted", UVM_LOW)

        //write_read_sequence
        `uvm_info("run_phase", "write read stimulus generation asserted", UVM_LOW)
        wr_rd_seq.start(w_env.agt.sqr);
        `uvm_info("run_phase", "write read stimulus generation deasserted", UVM_LOW)

        phase.drop_objection(this);
    endtask

endclass
endpackage
```

10) Env Package

```
package wrapper_env_pkg;
import uvm_pkg::*;
import wrapper_agent_pkg::*;
import wrapper_scoreboard_pkg::*;
import wrapper_cvr_pkg::*;
import wrapper_sequence_item_pkg::*;
`include "uvm_macros.svh"
class wrapper_env extends uvm_env;
    `uvm_component_utils(wrapper_env)
    wrapper_agent agt;
    wrapper_scoreboard sb;
    wrapper_coveragecollector cvr;
    wrapper_seq_item seq_item;
    function new(string name = "wrapper_env",uvm_component parent = null);
        super.new(name,parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt = wrapper_agent::type_id::create("agent",this);
        sb = wrapper_scoreboard::type_id::create("sb",this);
        cvr = wrapper_coveragecollector::type_id::create("cvr",this);
    endfunction
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.agt_ap.connect(sb.sb_export);
        agt.agt_ap.connect(cvr.cvr_export);
    endfunction
endclass
endpackage
```

11) Agent Package

```
package wrapper_agent_pkg;
import uvm_pkg::*;
import wrapper_driver_pkg::*;
import wrapper_monitor_pkg::*;
import wrapper_config_obj_pkg::*;
import wrapper_sequencer_pkg::*;
import wrapper_sequence_item_pkg::*;
`include "uvm_macros.svh"
class wrapper_agent extends uvm_agent;
    `uvm_component_utils(wrapper_agent)
    wrapper_config_obj cfg;
    wrapper_driver drv;
    wrapper_monitor mon;
    wrapper_sequencer sqr;
    uvm_analysis_port #(wrapper_seq_item) agt_ap;

    function new(string name = "wrapper_agent",uvm_component parent = null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(wrapper_config_obj)::get(this,"","W_CFG",cfg))
            `uvm_fatal("build phase","error")
        if(cfg.is_active == UVM_ACTIVE)begin
            sqr = wrapper_sequencer::type_id::create("sqr",this);
            drv = wrapper_driver::type_id::create("drv",this);
        end
        mon = wrapper_monitor::type_id::create("mon",this);
        agt_ap = new("agt_ap",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        if(cfg.is_active == UVM_ACTIVE)begin
            drv.wrapper_driver_vif = cfg.wrapper_config_vif;
            drv.seq_item_port.connect(sqr.seq_item_export);
        end
        mon.v_if = cfg.wrapper_config_vif;
        mon.mon_ap.connect(agt_ap);
    endfunction

endclass
endpackage
```


12) Rst Sequence

```
package wrapper_rst_sequence_pkg;
import shared_pkg::*;
import wrapper_sequence_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class wrapper_rst_seq extends uvm_sequence #(wrapper_seq_item);
`uvm_object_utils(wrapper_rst_seq)
    wrapper_seq_item seq_item;

    function new(string name = "wrapper_rst_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = wrapper_seq_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 0;
        seq_item.MOSI = 0;
        finish_item(seq_item);
    endtask
endclass
endpackage
```

13) Write Only Sequence

```
package wrapper_write_only_sequence_pkg;
import shared_pkg::*;
import wrapper_sequence_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class wrapper_write_only_seq extends uvm_sequence #(wrapper_seq_item);
`uvm_object_utils(wrapper_write_only_seq)
    wrapper_seq_item seq_item;

    function new(string name = "wrapper_write_only_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = wrapper_seq_item::type_id::create("seq_item");
        repeat(17000) begin
            start_item(seq_item);
            assert(seq_item.randomize() with {MOSI_bits[9:8] inside {[0:1]}});
            finish_item(seq_item);
        end
    endtask
endclass
endpackage
```

14) Read Only Sequence

```
package wrapper_read_only_sequence_pkg;
import shared_pkg::*;
import wrapper_sequence_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class wrapper_read_only_seq extends uvm_sequence #(wrapper_seq_item);
`uvm_object_utils(wrapper_read_only_seq)
    wrapper_seq_item seq_item;

    function new(string name = "wrapper_read_only_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = wrapper_seq_item::type_id::create("seq_item");
        seq_item.wr_c.constraint_mode(0); //closing to prevent conflict
        seq_item.rd_wr.constraint_mode(0); //closing to prevent conflict
        repeat(17000) begin
            start_item(seq_item);
            assert(seq_item.randomize() with {MOSI_bits[9:8] inside {[2:3]}});
            finish_item(seq_item);
        end
    endtask
endclass
endpackage
```

15) Read Write Sequence

```
package wrapper_write_read_sequence_pkg;
import wrapper_sequence_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class wrapper_write_read_seq extends uvm_sequence #(wrapper_seq_item);
`uvm_object_utils(wrapper_write_read_seq)
    wrapper_seq_item seq_item;

    function new(string name = "wrapper_write_read_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = wrapper_seq_item::type_id::create("seq_item");
        seq_item.rd_wr.constraint_mode(1); //reopen after closing
        seq_item.rd_c.constraint_mode(0);
        seq_item.wr_c.constraint_mode(0);
        repeat(17000) begin
            start_item(seq_item);
            assert(seq_item.randomize());
            finish_item(seq_item);
        end
    endtask
endclass
endpackage
```

16) Sequencer Package



```
package spi_sequencer_pkg;
import uvm_pkg::*;
import spi_sequence_item_pkg::*;
`include "uvm_macros.svh"
class spi_sequencer extends uvm_sequencer #(spi_seq_item);
    `uvm_component_utils(spi_sequencer)
    function new(string name = "spi_sequencer", uvm_component parent = null);
        super.new(name, parent);
    endfunction
endclass
endpackage
```

17) Sequence Item Package

```
package wrapper_sequence_item_pkg;
import uvm_pkg::*;
import shared_pkg::*;
`include "uvm_macros.svh"
class wrapper_seq_item extends uvm_sequence_item;
    `uvm_object_utils(wrapper_seq_item)
    rand logic rst_n;
    logic MOSI_SS_n,MISO,MISO_ref;
    rand bit [10:0] MOSI_bits;
    bit [10:0] MOSI_bits_old;
    static integer counter = 0;
    static integer bit_index = 10;
    static bit new_frame_needed = 1;
    op_t old_op;

    constraint rst_constrain {
        rst_n dist{0:=2,1:=98};
    }

    constraint mosi_constrain {
        MOSI_bits[10:8] inside {3'b000, 3'b001, 3'b110, 3'b111};
    }

    constraint wr_c {
        if(old_op == WRITE_ADDRESS)
        {
            MOSI_bits[10:8] inside {3'b000, 3'b001};
        }
    }

    constraint rd_c {
        if(old_op == READ_ADDRESS)
        {
            MOSI_bits[10:8] == 3'b111;
        }
        else if(old_op == READ_DATA)
        {
            MOSI_bits[10:8] == 3'b110;
        }
    }

    constraint rd_wr {
        if(old_op == WRITE_ADDRESS)
        {
            MOSI_bits[10:8] inside {3'b000, 3'b001};
        }
        if(old_op == READ_ADDRESS)
        {
            MOSI_bits[10:8] == 3'b111;
        }
        if(old_op == WRITE_DATA)
        {
            MOSI_bits[10:8] dist {3'b110:/60, 3'b000:/40};
        }
        else if(old_op == READ_DATA)
        {
            MOSI_bits[10:8] dist {3'b000:/60, 3'b110:/40};
        }
    }

    function new(string name = "wrapper_seq_item");
        super.new(name);
    endfunction
    function void pre_randomize();
        if(!rst_n)begin
            if(old_op == READ_DATA) old_op = READ_ADDRESS;
            else if(old_op == READ_ADDRESS) old_op = READ_DATA;
        end
    endfunction
    function void post_randomize();
        if(!rst_n)begin
            counter = 0;
            SS_n=1;
            new_frame_needed = 1;
            bit_index = 10;
        end
        else begin
            counter++;
            if (new_frame_needed) begin
                MOSI_bits_old = MOSI_bits;
                bit_index = 10;
                new_frame_needed = 0;
            end

            if(counter>=2)begin
                MOSI = MOSI_bits_old[bit_index];
                bit_index--;
            end

            if (bit_index < 0) begin
                bit_index = 10;
            end

            if (MOSI_bits_old[9:8] == 2'b11) begin
                if (counter == 23)begin
                    SS_n = 1;
                    counter = 0;
                    new_frame_needed = 1;
                end
            end
            else begin
                SS_n = 0;
            end
        end
        else begin
            if (counter == 14)begin
                SS_n = 1;
                counter = 0;
                new_frame_needed = 1;
            end
            else begin
                SS_n = 0;
            end
        end
        old_op = op_t'(MOSI_bits_old[9:8]);
    endfunction
endclass
```

```
function string convert2string();
    return $sformatf(
        "Cycle=%0d | rst_n=%0b | SS_n=%0b | MOSI=%0b | MISO=%0b | MOSI_bits=%0b",
        counter, rst_n, SS_n, MOSI, MISO, MOSI_bits);
endfunction

function string convert2string_stimulus();
    return $sformatf(
        "Cycle=%0d | rst_n=%0b | SS_n=%0b | MOSI=%0b",
        counter, rst_n, SS_n, MOSI);
endfunction
```

```
endclass
endpackage
```

18) Driver Package

```
package wrapper_driver_pkg;
import uvm_pkg::*;
import wrapper_sequence_item_pkg::*;
`include "uvm_macros.svh"
class wrapper_driver extends uvm_driver#(wrapper_seq_item);
    `uvm_component_utils(wrapper_driver)
    virtual wrapper_IF wrapper_driver_vif;
    wrapper_seq_item seq_item;

    function new(string name = "wrapper_driver",uvm_component parent = null);
        super.new(name,parent);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            seq_item = wrapper_seq_item :: type_id:: create("seq_item");
            seq_item_port.get_next_item(seq_item);
            wrapper_driver_vif.MOSI      = seq_item.MOSI;
            wrapper_driver_vif.rst_n     = seq_item.rst_n;
            wrapper_driver_vif.SS_n      = seq_item.SS_n;
            @(negedge wrapper_driver_vif.clk);
            seq_item_port.item_done();
            `uvm_info("run_phase",seq_item.convert2string_stimulus(),UVM_HIGH);
        end
    endtask

endclass
endpackage
```

19) Monitor Package

```
package wrapper_monitor_pkg;
import uvm_pkg::*;
import wrapper_sequence_item_pkg::*;
`include "uvm_macros.svh"
class wrapper_monitor extends uvm_monitor;
    `uvm_component_utils(wrapper_monitor)
    virtual wrapper_IF v_if;
    wrapper_seq_item seq_item;
    uvm_analysis_port #(wrapper_seq_item) mon_ap;

    function new(string name = "wrapper_monitor",uvm_component parent = null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon_ap = new("mon_ap",this);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            seq_item = wrapper_seq_item::type_id::create("seq_item");
            seq_item.MOSI      = v_if.MOSI;
            seq_item.rst_n     = v_if.rst_n;
            seq_item.SS_n      = v_if.SS_n;
            seq_item.MISO      = v_if.MISO;
            seq_item.MISO_ref  = v_if.MISO_ref;
            @(negedge v_if.clk);
            mon_ap.write(seq_item);
        end
    endtask
endclass
endpackage
```

20) Scoreboard Package

```
package wrapper_scoreboard_pkg;
import uvm_pkg::*;
import wrapper_sequence_item_pkg::*;
`include "uvm_macros.svh"

class wrapper_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(wrapper_scoreboard)

    wrapper_seq_item seq_item;
    uvm_analysis_export #(wrapper_seq_item) sb_export;
    uvm_tlm_analysis_fifo #(wrapper_seq_item) sb_fifo;

    int correct_counter = 0;
    int error_counter = 0;

    function new(string name = "wrapper_scoreboard", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export", this);
        sb_fifo = new("sb_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            sb_fifo.get(seq_item);
            if (seq_item.MISO != seq_item.MISO_ref)
                error_counter++;
            else
                correct_counter++;
        end
    endtask

    function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        $display("Final Counters for wrapper %d,%d", correct_counter, error_counter);
    endfunction

endclass
endpackage
```

21) Coverage Package

- Coverage Being Fullfilled in the Coverage package of the SPI_Slave and Ram

```
package wrapper_cvr_pkg;
import uvm_pkg::*;
import wrapper_sequence_item_pkg::*;
`include "uvm_macros.svh"
class wrapper_coveragecollector extends uvm_component;
    `uvm_component_utils(wrapper_coveragecollector)
    wrapper_seq_item seq_item;
    uvm_analysis_export #(wrapper_seq_item) cvr_export;
    uvm_tlm_analysis_fifo #(wrapper_seq_item) cvr_fifo;

    function new(string name = "wrapper_coveragecollector",uvm_component parent = null);
        super.new(name,parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        cvr_export = new("cvr_export",this);
        cvr_fifo = new("cvr_fifo",this);
    endfunction
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        cvr_export.connect(cvr_fifo.analysis_export);
    endfunction
    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cvr_fifo.get(seq_item);
        end
    endtask
endclass
endpackage
```


22) Do File / Src File

```
vlib work

vlog -f src_files.list +cover +define+SIM -covercells

vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover

coverage save wrapper.ucdb -onexit

add wave /top/w_vif/*

add wave /top/s_vif/*

add wave /top/r_vif/*

run -all

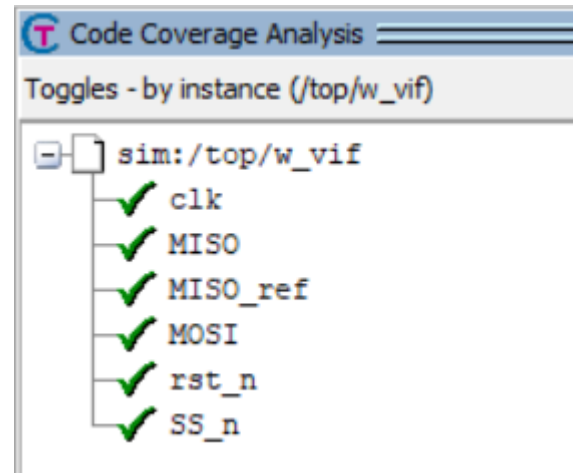
quit -sim

vcover report wrapper.ucdb -details -annotate -all -output wrapper.txt
```

```
shared_pkg.sv
spi_if.sv
ram_if.sv
wrapper_if.sv
spi_sva.sv
ram_sva.sv
wrapper_sva.sv
SPI_golden.v
RAM_golden.v
SPI_WRAPPER_GOLDEN.v
SPI_slave.v
RAM.v
SPI_wrapper.v
spi_config_obj.sv
ram_config_obj.sv
wrapper_config_obj.sv
spi_sequence_item.sv
ram_sequence_item.sv
wrapper_sequence_item.sv
wrapper_rst_sequence.sv
wrapper_read_only_seq.sv
wrapper_write_only_seq.sv
wrapper_write_read_seq.sv
spi_sequencer.sv
ram_sequencer.sv
wrapper_sequencer.sv
spi_driver.sv
ram_driver.sv
wrapper_driver.sv
spi_monitor.sv
ram_monitor.sv
wrapper_monitor.sv
spi_sb.sv
ram_sb.sv
wrapper_sb.sv
spi_cvr.sv
ram_cvr.sv
wrapper_cvr.sv
spi_agent.sv
ram_agent.sv
wrapper_agent.sv
spi_env.sv
ram_env.sv
wrapper_env.sv
wrapper_test.sv
top.sv
```

23) Toggle Coverage

Since in the Wrapper Design there is no Branch or Statements of FSM so the only Coverage Report is the toggle but SPI and RAM coverage reports provided up



24) Functional Coverage

Covergroups									
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_ins	
[-] /ram_coverage_pkg/ram_coverage		100.00%							
[-] + TYPE Covram		100.00%	100	100.00...	✓	✓	auto(0)		
[-] /spi_cvr_pkg/spi_coveragecollector		100.00%							
[-] + TYPE cvr_grp		100.00%	100	100.00...	✓	✓	auto(0)		

25) Assertion

▲ /wrapper_write_read_sequence_pkg::wrapper_write_read_seq::body/#...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓	
▲ /wrapper_read_only_sequence_pkg::wrapper_read_only_seq::body/#u...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓	
▲ /wrapper_write_only_sequence_pkg::wrapper_write_only_seq::body/#...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓	
⊕ /top/W_DUT/RAM_instance/ram_inst/areset	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge r_vif.clk) (~r_vif...	✓
⊕ /top/W_DUT/RAM_instance/ram_inst/abx_valid_j	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge r_vif.clk) disabli...	✓
⊕ /top/W_DUT/RAM_instance/ram_inst/abx_valid_h	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge r_vif.clk) disabli...	✓
⊕ /top/W_DUT/RAM_instance/ram_inst/awr_addr_data	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge r_vif.clk) disabli...	✓
⊕ /top/W_DUT/RAM_instance/ram_inst/ard_addr_data	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge r_vif.clk) disabli...	✓
⊕ /top/W_DUT/SLAVE_instance/spi_inst/assert1	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge s_vif.clk) (~s_vif...	✓
⊕ /top/W_DUT/SLAVE_instance/spi_inst/assert2	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge s_vif.clk) disabli...	✓
⊕ /top/W_DUT/wrapper_inst/assert1	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge w_vif.clk) (~w_vif...	✓
⊕ /top/W_DUT/wrapper_inst/assert2	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge w_vif.clk) disab...	✓

26) Assertion Coverage

Cover Directives															
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	
▲ /top/W_DUT/RAM_instance/ram_inst/creset	SVA	✓	Off	1057	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	
▲ /top/W_DUT/RAM_instance/ram_inst/ctx_valid_j	SVA	✓	Off	4169	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	
▲ /top/W_DUT/RAM_instance/ram_inst/ctx_valid_h	SVA	✓	Off	1826	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	
▲ /top/W_DUT/RAM_instance/ram_inst/cwr_addr_d...	SVA	✓	Off	12400	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	
▲ /top/W_DUT/RAM_instance/ram_inst/crd_addr_da...	SVA	✓	Off	7180	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	
▲ /top/W_DUT/SLAVE_instance/spi_inst/cover1	SVA	✓	Off	1057	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	
▲ /top/W_DUT/SLAVE_instance/spi_inst/cover2	SVA	✓	Off	1316	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	
▲ /top/W_DUT/wrapper_inst/cover1	SVA	✓	Off	1057	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	
▲ /top/W_DUT/wrapper_inst/cover2	SVA	✓	Off	1962	1	Unli...	1	100%	✓	✓	0	0	0 ns	0	

Assertions Table

SPI Assertions

Feature	Assertion
whenever reset is asserted, (MISO, rx_valid, and rx_data) are all low.	@(posedge s_vif.clk) !s_vif.rst_n => (!s_vif.MISO && !s_vif.rx_valid && ~s_vif.rx_data);
any valid command sequence (write_add_seq (000), write_data_seq (001), read_add_seq(110), or read_data_seq(111)), the rx_valid signal must assert exactly after 10 cycles and the SS_n should eventually after the 10 cycles to close communication.	@(posedge s_vif.clk) disable iff(!s_vif.rst_n) ((write_addr_sequence) or (write_data_sequence) or (read_addr_sequence) or (read_data_sequence -> rx_ssn_sequence));
assertions in the design to check correct FSM transition: IDLE → CHK_CMD	@(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) (cs == IDLE) -> (ns == CHK_CMD);
assertions in the design to check correct FSM transition: CHK_CMD → WRITE or READ_ADD or READ_DATA	@(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) (cs == CHK_CMD) -> (ns == WRITE ns == READ_DATA ns == READ_ADD);
assertions in the design to check correct FSM transition: WRITE → IDLE	@(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) (\$past(\$past(\$past(spi_vif.SS_n))) && cs == WRITE) -> ##12 (cs == IDLE);
assertions in the design to check correct FSM transition: READ_ADD → IDLE	@(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) (\$past(\$past(\$past(spi_vif.SS_n))) && cs == READ_ADD) -> ##12 (cs == IDLE);
assertions in the design to check correct FSM transition: READ_DATA → IDLE	@(posedge spi_vif.clk) disable iff(!spi_vif.rst_n) (\$past(\$past(\$past(spi_vif.SS_n))) && cs == READ_DATA) -> ##21 (cs == IDLE);

RAM Assertions

Feature	Assertion
whenever reset is asserted, tx_valid and dout are low	<pre>@(posedge r.clk) !r.rst_n => (r.dout == 0 && r.tx_valid == 0);</pre>
during (write_add_seq, write_data_seq, read_add_seq) the tx_valid signal must remain deasserted.	<pre>@(posedge r.clk) disable iff(!r.rst_n) (r.din[9:8] == 2'b00 r.din[9:8] == 2'b01 r.din[9:8] == 2'b10) && (r.rx_valid == 1'b1) => r.tx_valid == 0 ;</pre>
after a read_data_seq , the tx_valid signal must rise and after it rises by one clock cycle, it should eventually fall.	<pre>@(posedge r.clk) disable iff(!r.rst_n) (r.din[9:8] == 2'b11) && (r.rx_valid == 1'b1) => \$rose(r.tx_valid) ##1 \$fell(r.tx_valid) ;</pre>
every Write Address operation must be eventually followed by a Write Data operation.	<pre>@(posedge r.clk) disable iff(!r.rst_n) (r.din[9:8] == 2'b00) -> ##[1:\$] (r.din[9:8] == 2'b01) ;</pre>
every Read Address operation must be eventually followed by a Read Data operation.	<pre>@(posedge r.clk) disable iff(!r.rst_n) (r.din[9:8] == 2'b10) -> ##[1:\$] (r.din[9:8] == 2'b11) ;</pre>

Wrapper Assertion

Feature	Assertion
whenever reset is asserted, the outputs (MISO, rx_valid, and rx_data) are all inactive.	<pre>@(posedge w_vif.clk) !w_vif.rst_n => (!w_vif.MISO);</pre>
assertion to make sure that the MISO remains with a stable value as long as it is not a read data operation	<pre>@(posedge w_vif.clk) disable iff(!w_vif.rst_n) (write_addr_sequence or write_data_sequence or read_addr_sequence) -> \$stable(w_vif.MISO);</pre>